Assessment and Answer Guide

DEVOPS ENGINEERING MASTERY — Volume 15, Companion Volume. First Edition, 9 November 2026. This guide contains 120 MCQs, 30 short-answer questions, 20 scenario-based troubleshooting questions, 10 incident case studies, the practical final examination and the capstone rubric — followed by a separate answer key with explanations. Attempt every question yourself before reading its answer; the explanation matters more than the letter.

> **WARNING:** Do not copy assignment solutions word for word. Read the outline, close this guide, and do the work yourself in your own lab environment. That is where the learning happens.

মূল্যায়ন ও দ্রুত রেফারেন্স সহায়িকা • Volume 15

মূল্যায়ন ও দ্রুত রেফারেন্স সহায়িকা — ভূমিকা

এই সহায়িকাটি **DevSecOps, SRE ও Production Reliability Engineering** বইয়ের ২৪টি অধ্যায়ের শেখা যাচাই ও কাজে প্রয়োগের জন্য তৈরি। এটি দুটি কাজে ব্যবহৃত হবে — একটি **Assessment Guide** হিসেবে (পরীক্ষা, অনুশীলন ও ক্যাপস্টোন মূল্যায়ন) এবং একটি **Quick Reference Handbook** হিসেবে (কমান্ড, চেকলিস্ট, ওয়ার্কফ্লো ও ডিসিশন ট্রি)।

এই বইয়ের মূল দর্শন একটাই: **নিরাপত্তা ও নির্ভরযোগ্যতা কোনো শেষ ধাপ নয় — এগুলো ডিজাইন, কোড, পাইপলাইন, ডেপ্লয়মেন্ট ও অপারেশনের প্রতিটি স্তরে বুনে দিতে হয়।** তাই প্রশ্নগুলো শুধু সংজ্ঞা মুখস্থ যাচাই করে না; বাস্তব প্রোডাকশন পরিস্থিতিতে আপনি কী সিদ্ধান্ত নেবেন, কী অগ্রাধিকার দেবেন এবং কী ট্রেড-অফ মেনে নেবেন — সেটাই যাচাই করে।

১. এই সহায়িকা কীভাবে ব্যবহার করবেন

- **MCQ (১–১২০):** প্রতিটি অধ্যায় শেষে নিজেকে যাচাই করুন। ভুল উত্তরের ব্যাখ্যা পড়ে ধারণাটি আবার দেখুন — শুধু সঠিক অক্ষরটি মনে রাখবেন না।
- **সংক্ষিপ্ত প্রশ্ন (৩০টি):** ২–৪ বাক্যে উত্তর লেখার অনুশীলন — ইন্টারভিউ ও ভাইভার জন্য উপযোগী।
- **পরিস্থিতি (২০টি) ও কেস স্টাডি (১০টি):** দলগত আলোচনা, টেবিল-টপ এক্সারসাইজ ও ইনসিডেন্ট রিভিউ অনুশীলনের জন্য।
- **ফাইনাল পরীক্ষা (১০০ নম্বর, ~৩ ঘণ্টা):** পুরো বইয়ের সামগ্রিক মূল্যায়ন।
- **গ্লসারি (১৮০টি টার্ম):** বাংলা ব্যাখ্যাসহ দ্রুত রেফারেন্স।
- **কমান্ড, ওয়ার্কফ্লো, চেকলিস্ট ও ডিসিশন ট্রি:** প্রোডাকশনে কাজ করার সময় হাতের কাছে রাখার মতো অংশ।

২. মূল্যায়নের নীতি

- সব অনুশীলন **অনুমোদিত, আইসোলেটেড (বিচ্ছিন্ন) ল্যাব পরিবেশে** করবেন। অন্যের সিস্টেমে অনুমতি ছাড়া স্ক্যান, টেস্ট বা আক্রমণমূলক কাজ কখনোই করবেন না।
- কোনো বাস্তব সিক্রেট (পাসওয়ার্ড, টোকেন, প্রাইভেট কি) এই বইয়ের উদাহরণে ব্যবহার করবেন না — শুধু `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করুন।

- উত্তর লেখার সময় **কেন** — অর্থাৎ যুক্তি ও ট্রেড-অফ — ব্যাখ্যা করাকে নম্বরের অংশ হিসেবে ধরা হয়।
- পাস মার্ক মানে মুখস্থ নয় — মানে আপনি একটি নিরাপদ, পর্যবেক্ষণযোগ্য ও পুনরুদ্ধারযোগ্য সিস্টেম ডিজাইন ও পরিচালনা করতে পারেন।

কীভাবে পড়বেন: প্রথমে MCQ দিয়ে দুর্বল জায়গা চিহ্নিত করুন, তারপর সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন, তারপর পরিস্থিতি ও কেস স্টাডি দিয়ে প্রয়োগ করুন। শেষে ফাইনাল পরীক্ষা দিয়ে নিজেকে যাচাই করুন।

সতর্কতা: এই বইয়ের নিরাপত্তা টুল ও কৌশল শুধু শেখা ও প্রতিরক্ষার জন্য। অনুমোদনহীন অ্যাক্সেস, ডেটা চুরি বা সিস্টেমের ক্ষতি করা আইনত দণ্ডনীয় ও নৈতিকভাবে অগ্রহণযোগ্য।

অধ্যায় ১–২৪ • ১২০টি প্রশ্ন

বহুনির্বাচনী প্রশ্ন (MCQ) — ১২০টি

প্রতিটি প্রশ্নের চারটি বিকল্প আছে — ক, খ, গ, ঘ। সঠিক উত্তরটি বেছে নিন। কঠিনতা: [Beginner] মৌলিক ধারণা, [Intermediate] প্রয়োগ ও তুলনা, [Advanced] ট্রেড-অফ ও ইঞ্জিনিয়ারিং বিচার। উত্তরমালা answer-guide ও answer-guide-2 অংশে দেওয়া আছে — আগে নিজে চেষ্টা করুন।

১. DevSecOps ও নিরাপত্তা-ভিত্তি

১. DevSecOps-এর মূল ধারণা কোনটি? [Beginner]

ক) নিরাপত্তাকে ডেভেলপমেন্ট ও অপারেশনের শুরু থেকেই প্রতিটি ধাপে যুক্ত করা — shift-left
খ) ডেপ্লয়ের পরে শুধু একবার নিরাপত্তা অডিট করা
গ) নিরাপত্তা দলের কাজ শুধু ফায়ারওয়াল দেখা
ঘ) দ্রুত ডেলিভারির জন্য নিরাপত্তা পরীক্ষা বাদ দেওয়া

২. Threat modeling সবচেয়ে কার্যকর হয় কখন? [Intermediate]

ক) প্রোডাকশনে ইনসিডেন্ট হওয়ার পর
খ) ডিজাইন পর্যায়ে, আর্কিটেকচার ঠিক হওয়ার আগেই হুমকি ও প্রতিরোধ চিহ্নিত করলে
গ) শুধু পেনিট্রেশন টেস্টের সময়
ঘ) অডিটের আগের রাতে

৩. STRIDE মডেলে "T" কী বোঝায়? [Beginner]

ক) Testing
খ) Tracking
গ) Tampering — ডেটা বা কোডের অননুমোদিত পরিবর্তন
ঘ) Timeout

৪. একটি পেমেন্ট API ডিজাইন করছেন। সবচেয়ে আগে কোন হুমকিটি বিবেচনা করবেন? [Advanced]

ক) লোগোর রং
খ) API ডকুমেন্টেশনের ফন্ট
গ) সার্ভারের ব্র্যান্ড
ঘ) প্রমাণীকরণ (authentication) ও অনুমোদন (authorization) বাইপাস এবং সংবেদনশীল ডেটা ফাঁস

৫. OWASP Top 10: 2021-এর A01:2021 কোনটি? [Beginner]

ক) Cryptographic Failures
খ) Security Misconfiguration
গ) Broken Access Control
ঘ) Injection

৬. Broken Access Control প্রতিরোধে সবচেয়ে নির্ভরযোগ্য পদ্ধতি কোনটি? [Intermediate]

ক) সার্ভার-সাইডে প্রতিটি রিকোয়েস্টে অনুমোদন যাচাই করা (deny by default)
খ) শুধু ফ্রন্টএন্ডে বোতাম লুকিয়ে রাখা
গ) ইউজারকে বিশ্বাস করা
ঘ) URL গোপন রাখা (security by obscurity)

৭. SQL Injection প্রতিরোধের সবচেয়ে ভালো উপায় কোনটি? [Beginner]

ক) ইউজার ইনপুট সরাসরি কোয়েরিতে বসানো
খ) শুধু ইনপুটের দৈর্ঘ্য সীমিত করা
গ) Error message লুকিয়ে রাখা
ঘ) Parameterized query / prepared statement ব্যবহার করা

৮. একটি লিগ্যাসি অ্যাপে একসাথে SQLi, XSS ও Broken Access Control পাওয়া গেছে। সীমিত সময়ে প্রথমে কোনটি ঠিক করবেন? [Advanced]

ক) যেটি ঠিক করা সবচেয়ে সহজ
খ) যেটির শোষণ (exploitability) ও প্রভাব (impact) সবচেয়ে বেশি — সাধারণত প্রমাণীকরণ/অনুমোদন বাইপাস ও SQLi-কে অগ্রাধিকার দিয়ে ঝুঁকি অনুযায়ী সাজানো
গ) যেটি রিপোর্টে প্রথমে লেখা আছে
ঘ) শুধু XSS, কারণ তা চোখে দেখা যায়

৯. XSS প্রতিরোধে কোনটি সবচেয়ে কার্যকর? [Intermediate]

ক) ইনপুটে শুধু <script> শব্দটি ব্লক করা
খ) আউটপুট এনকোডিং + Content Security Policy (CSP) + প্রসঙ্গ-সচেতন (context-aware) এস্কেপিং
গ) ব্রাউজার আপডেট করতে বলা
ঘ) JavaScript বন্ধ করে দেওয়া

১০. SSRF (Server-Side Request Forgery) আক্রমণে আক্রমণকারী কী করে? [Intermediate]

ক) ডেটাবেস মুছে ফেলে
খ) শুধু ফ্রন্টএন্ডের CSS বদলায়
গ) সার্ভারকে দিয়ে অভ্যন্তরীণ (internal) সার্ভিস বা মেটাডেটা এন্ডপয়েন্টে রিকোয়েস্ট পাঠায়
ঘ) পাসওয়ার্ড অনুমান করে

১১. সিক্রেট (API key, পাসওয়ার্ড) গিট রিপোজিটরিতে কমিট হয়ে গেলে প্রথম পদক্ষেপ কী? [Advanced]

ক) কমিট মেসেজ বদলে দেওয়া
খ) ফাইলটি .gitignore-এ যোগ করা
গ) রিপোজিটরি প্রাইভেট করা
ঘ) সিক্রেটটি সঙ্গে সঙ্গে revoke/rotate করা, ইতিহাস থেকে সরানো এবং লিকের প্রভাব যাচাই করা

১২. SAST (Static Application Security Testing) কী পরীক্ষা করে? [Beginner]

ক) সোর্স কোড না চালিয়ে কোডের ভেতরের দুর্বলতা
খ) নেটওয়ার্ক পোর্ট
গ) ইউজারের আচরণ
ঘ) চলমান অ্যাপের HTTP রেসপন্স

১৩. DAST কখন সবচেয়ে মূল্যবান? [Intermediate]

ক) শুধু ইউনিট টেস্টের বদলে
খ) ডকুমেন্টেশন লেখার সময়
গ) কোড লেখার আগেই
ঘ) চলমান (running) অ্যাপ্লিকেশনকে বাইরে থেকে পরীক্ষা করে রানটাইম দুর্বলতা ও ভুল কনফিগারেশন ধরতে

১৪. SCA (Software Composition Analysis) মূলত কী খুঁজে বের করে? [Beginner]

ক) কোডের লাইন সংখ্যা
খ) ব্যবহৃত থার্ড-পার্টি লাইব্রেরি ও ডিপেন্ডেন্সিতে জানা CVE দুর্বলতা ও লাইসেন্স ঝুঁকি
গ) সার্ভারের CPU ব্যবহার
ঘ) ইউজার ইন্টারফেসের বাগ

১৫. CI পাইপলাইনে SAST স্ক্যানে অনেক false positive আসছে। ডেভেলপাররা স্ক্যান এড়াতে চাইছে। সঠিক পদক্ষেপ কোনটি? [Advanced]

ক) নিয়ম (rules) টিউন করা, উচ্চ-আস্থার (high-confidence) নিয়মে ব্লক করা, বাকিগুলো warning হিসেবে রাখা এবং ডেভেলপারদের সাথে ত্রিয়াজ করা
খ) সব finding উপেক্ষা করা
গ) শুধু রিলিজের দিন স্ক্যান চালানো
ঘ) SAST পুরো বন্ধ করে দেওয়া

১৬. SBOM (Software Bill of Materials) কেন গুরুত্বপূর্ণ? [Intermediate]

ক) এটি লাইসেন্স ফি কমায়
খ) এটি কোড সুন্দর করে
গ) সফটওয়্যারের প্রতিটি উপাদান ও সংস্করণের তালিকা দেয়, ফলে নতুন CVE এলে দ্রুত জানা যায় কোন সার্ভিস আক্রান্ত
ঘ) এটি ডেপ্লয় দ্রুত করে

১৭. SLSA ফ্রেমওয়ার্কের মূল লক্ষ্য কী? [Intermediate]

ক) ডেটাবেস অপ্টিমাইজ করা
খ) কোডের গতি বাড়ানো
গ) সফটওয়্যার সাপ্লাই চেইনের অখণ্ডতা (integrity) নিশ্চিত করা — বিল্ডের উৎস, প্রক্রিয়া ও আর্টিফ্যাক্ট যাচাইযোগ্য করা
ঘ) UI ডিজাইন করা

১৮. কনটেইনার ইমেজে critical CVE পাওয়া গেলে সবচেয়ে ভালো প্রতিক্রিয়া কোনটি? [Advanced]

ক) শুধু ট্যাগের নাম বদলে দেওয়া
খ) CVE নম্বরটি লুকিয়ে রাখা
গ) ইমেজটি প্রোডাকশনে পাঠিয়ে পরে দেখা
ঘ) বেস ইমেজ ও ডিপেন্ডেন্সি আপডেট করে আবার বিল্ড ও স্ক্যান করা, এবং শোষণযোগ্যতা (exploitability) যাচাই করে অগ্রাধিকার ঠিক করা

১৯. Docker ইমেজের আকার ও আক্রমণ-ক্ষেত্র (attack surface) কমাতে কোনটি ভালো? [Beginner]

ক) Multi-stage build + minimal/distroless বেস ইমেজ + অপ্রয়োজনীয় প্যাকেজ বাদ দেওয়া
খ) সব টুল ইমেজে রাখা

গ) root ইউজার হিসেবে চালানো
ঘ) বড় বেস ইমেজ ব্যবহার করা

২০. Kubernetes-এ Pod-কে root হিসেবে চলতে না দেওয়ার সঠিক উপায় কোনটি? [Intermediate]

ক) Pod-এর নাম বদলে দেওয়া
খ) securityContext-এ runAsNonRoot: true ও readOnlyRootFilesystem: true দেওয়া এবং Pod Security Standards প্রয়োগ করা
গ) Namespace-এর নাম লুকানো
ঘ) শুধু ইমেজ ট্যাগ latest ব্যবহার করা

২১. Kubernetes RBAC-এ least privilege মানে কী? [Intermediate]

ক) সবাইকে cluster-admin দেওয়া
খ) সব ServiceAccount-কে একই ভূমিকা দেওয়া
গ) Namespace মুছে ফেলা
ঘ) প্রতিটি ব্যবহারকারী ও ServiceAccount-কে শুধু তার কাজের জন্য প্রয়োজনীয় ন্যূনতম অনুমতি দেওয়া

২২. Kubernetes Secret সরাসরি env variable হিসেবে ব্যবহারের ঝুঁকি কী? [Intermediate]

ক) Secret-কে base64-এ এনকোড করে etcd-তে রাখা হয় — ডিফল্টে এনক্রিপ্টেড নয়, এবং env থেকে লগে ফাঁস হতে পারে; তাই external secrets manager ও encryption at rest ব্যবহার করা উচিত
খ) Secret ব্যবহার করলেই অ্যাপ ধীর হয়
গ) Secret-এর নাম বড় হয়
ঘ) Secret শুধু YAML-এ লেখা যায় না

২৩. NetworkPolicy না থাকলে Kubernetes ক্লাস্টারে কী হয়? [Beginner]

ক) DNS বন্ধ হয়ে যায়
খ) সব Pod একে অপরের সাথে অবাধে কথা বলতে পারে — lateral movement সহজ হয়
গ) Pod চলা বন্ধ হয়
ঘ) Ingress কাজ করে না

২৪. OPA / Kyverno-এর মতো Policy as Code টুলের কাজ কী? [Intermediate]

ক) কোড ফরম্যাট করা
খ) লগ সংগ্রহ করা
গ) ডেপ্লয়মেন্টের আগে নীতি (যেমন: latest ট্যাগ নিষেধ, resource limit বাধ্যতামূলক) যাচাই করে নীতি-বিরোধী কাজ আটকে দেওয়া
ঘ) ডেটাবেস ব্যাকআপ নেওয়া

২৫. CVSS স্কোর ৯.৮-এর অর্থ কী? [Beginner]

ক) Critical তীব্রতার দুর্বলতা — সহজে শোষণযোগ্য ও ব্যাপক প্রভাব
খ) কম ঝুঁকির দুর্বলতা
গ) শুধু তথ্যগত (informational) সমস্যা
ঘ) স্কোরটি ভুল

২৬. CVSS স্কোর বেশি হলেই কি সবসময় প্রথমে ঠিক করতে হবে? [Advanced]

ক) হ্যাঁ, স্কোরই একমাত্র মাপকাঠি
খ) না, শুধু সংখ্যা দেখলেই হয়
গ) স্কোর দেখে সব ঠিক করা যায়
ঘ) না — শোষণযোগ্যতা (exploitability), সম্পদের এক্সপোজার, ব্যবসায়িক প্রভাব ও compensating control মিলিয়ে ঝুঁকি-ভিত্তিক অগ্রাধিকার ঠিক করতে হয়

২৭. Vulnerability management-এর সঠিক জীবনচক্রের ক্রম কোনটি? [Intermediate]

ক) ঠিক করা → আবিষ্কার → রিপোর্ট
খ) আবিষ্কার (discover) → ত্রিয়াজ ও অগ্রাধিকার → প্রতিকার (remediate) → যাচাই (verify) → পুনরাবৃত্তি
গ) উপেক্ষা করা → ভুলে যাওয়া
ঘ) রিপোর্ট লেখা → আবিষ্কার

২৮. ক্লাউডে S3 বাকেট public হয়ে গেলে প্রথমে কী করবেন? [Intermediate]

ক) বাকেটের নাম বদলে দেওয়া
খ) অপেক্ষা করা
গ) অবিলম্বে public অ্যাক্সেস বন্ধ করা, কে কী অ্যাক্সেস করেছে তা লগ থেকে যাচাই করা এবং Block Public Access ও SCP/নীতি দিয়ে পুনরাবৃত্তি রোধ করা
ঘ) শুধু টিমকে ইমেইল করা

২৯. CSPM (Cloud Security Posture Management) টুল কী করে? [Intermediate]

ক) ক্লাউড বিল কমায়
খ) ক্লাউড কনফিগারেশন ক্রমাগত স্ক্যান করে ভুল কনফিগারেশন, অতিরিক্ত অনুমতি ও কমপ্লায়েন্স বিচ্যুতি ধরিয়ে দেয়
গ) শুধু VM তৈরি করে
ঘ) DNS পরিচালনা করে

৩০. IAM-এ সবচেয়ে নিরাপদ অভ্যাস কোনটি? [Beginner]

ক) Least privilege + MFA + স্বল্পমেয়াদি (short-lived) ক্রেডেনশিয়াল ও নিয়মিত অ্যাক্সেস রিভিউ
খ) Root অ্যাকাউন্ট দৈনন্দিন কাজে ব্যবহার করা
গ) দীর্ঘমেয়াদি অ্যাক্সেস কি সবার সাথে ভাগ করা
ঘ) একই পাসওয়ার্ড সব জায়গায় ব্যবহার করা

৩১. CI/CD পাইপলাইনে সিক্রেট নিরাপদে ব্যবহারের সঠিক উপায় কোনটি? [Intermediate]

ক) সিক্রেট .env ফাইল হিসেবে গিটে কমিট করা
খ) সিক্রেট CI লগে প্রিন্ট করা
গ) সিক্রেটকে মাস্কড ভেরিয়েবল বা secrets manager থেকে রানটাইমে আনা, লগে না দেখানো এবং স্বল্পমেয়াদি টোকেন ব্যবহার করা
ঘ) সিক্রেট Slack-এ পোস্ট করা

৩২. সাপ্লাই চেইন আক্রমণ থেকে বাঁচতে কোনটি সবচেয়ে কার্যকর? [Advanced]

ক) সব ডিপেন্ডেন্সি latest ব্যবহার করা
খ) ডিপেন্ডেন্সি যাচাই না করা

গ) অজানা সোর্স থেকে প্যাকেজ নেওয়া
ঘ) পিন করা সংস্করণ + লকফাইল + SBOM + স্বাক্ষর যাচাই (Sigstore/cosign) + নির্ভরযোগ্য রেজিস্ট্রি

৩৩. GitHub Actions-এ pull_request-এ GITHUB_TOKEN-এর অনুমতি সীমিত রাখা কেন জরুরি? [Advanced]

ক) টোকেন লম্বা হয় বলে
খ) CI দ্রুত চলে বলে
গ) লগ সুন্দর হয় বলে
ঘ) আক্রমণকারী PR-এর মাধ্যমে টোকেন চুরি করে রিপোজিটরি বা রিলিজে লেখার চেষ্টা করতে পারে — তাই ন্যূনতম (least privilege) অনুমতি দিতে হয়

৩৪. Cosign দিয়ে কনটেইনার ইমেজ স্বাক্ষর (sign) করার উদ্দেশ্য কী? [Intermediate]

ক) ইমেজ ছোট করা
খ) ইমেজ দ্রুত টানা
গ) ইমেজটি নির্ভরযোগ্য উৎস থেকে এসেছে ও পথে বদলায়নি — তা যাচাইযোগ্য করা
ঘ) ইমেজের রং বদলানো

৩৫. Zero Trust মডেলের মূল নীতি কোনটি? [Beginner]

ক) কাউকে ডিফল্টে বিশ্বাস নয় — প্রতিটি অ্যাক্সেস যাচাই করো (never trust, always verify)
খ) ভেতরের নেটওয়ার্ককে অন্ধভাবে বিশ্বাস করা
গ) শুধু VPN থাকলেই যথেষ্ট
ঘ) পাসওয়ার্ডই একমাত্র নিরাপত্তা

৩৬. SRE (Site Reliability Engineering)-এর মূল লক্ষ্য কী? [Beginner]

ক) যত বেশি ফিচার তত ভালো
খ) সফটওয়্যার ইঞ্জিনিয়ারিং দিয়ে অপারেশন সমস্যার সমাধান করে নির্ভরযোগ্য সেবা দেওয়া
গ) শুধু সার্ভার কেনা
ঘ) ম্যানুয়াল কাজ বাড়ানো

৩৭. SLI (Service Level Indicator) কী? [Beginner]

ক) সেবার মানের পরিমাপযোগ্য সূচক — যেমন সফল রিকোয়েস্টের অনুপাত বা লেটেন্সি
খ) সেবার দাম
গ) সেবার নাম
ঘ) সেবার লোগো

৩৮. SLO ৯৯.৯%-এর অর্থ কী? [Intermediate]

ক) মাসে ৯৯.৯% সময় সার্ভার বন্ধ থাকবে
খ) নির্ধারিত সময়ে ৯৯.৯% রিকোয়েস্ট সফলভাবে লক্ষ্য মেনে চলতে হবে — যেমন লেটেন্সি সীমার মধ্যে
গ) ৯৯.৯% ইউজার সন্তুষ্ট
ঘ) ৯৯.৯% কোড কভারেজ

৩৯. Error budget কী? [Intermediate]

ক) বাগ ঠিক করার টাকা
খ) ডেভেলপারদের বেতন
গ) মনিটরিং টুলের খরচ
ঘ) SLO-র অনুমোদিত ব্যর্থতার পরিমাণ (১০০% − SLO) — এই বাজেট শেষ হলে নতুন ফিচার থামিয়ে নির্ভরযোগ্যতায় মনোযোগ দিতে হয়

৪০. Error budget শেষ হয়ে গেলে সঠিক সিদ্ধান্ত কোনটি? [Advanced]

ক) মনিটরিং বন্ধ করে দেওয়া
খ) SLO কমিয়ে দেওয়া যাতে বাজেট ফিরে আসে
গ) ফিচার ফ্রিজ ঘোষণা করে নির্ভরযোগ্যতা, টেকনিক্যাল ঋণ ও স্থিতিশীলতায় বিনিয়োগ করা
ঘ) তবুও ফিচার রিলিজ চালিয়ে যাওয়া

৪১. কোন SLI-টি availability-র জন্য সবচেয়ে উপযুক্ত? [Intermediate]

ক) CPU ব্যবহারের হার
খ) সফল রিকোয়েস্টের অনুপাত (successful requests / total requests)
গ) লগের আকার
ঘ) ডেপ্লয়ের সংখ্যা

৪২. SLO নির্ধারণের সময় সবচেয়ে গুরুত্বপূর্ণ বিবেচনা কোনটি? [Advanced]

ক) ব্যবহারকারীর প্রত্যাশা ও ব্যবসায়িক প্রভাবের সাথে সামঞ্জস্য — খুব কঠিন SLO অপ্রয়োজনীয় খরচ বাড়ায়, খুব ঢিলা SLO ব্যবহারকারী হারায়
খ) প্রতিযোগীর SLO কপি করা
গ) ১০০% SLO নির্ধারণ করা
ঘ) শুধু ইঞ্জিনিয়ারদের সুবিধা দেখা

৪৩. Golden Signals-এ চারটি সংকেত কোনগুলো? [Beginner]

ক) CPU, মেমোরি, ডিস্ক, নেটওয়ার্ক
খ) লগ, ট্রেস, মেট্রিক, ইভেন্ট
গ) Error, warning, info, debug
ঘ) Latency, Traffic, Errors, Saturation

৪৪. RED মেথড (Rate, Errors, Duration) কীসের জন্য ব্যবহৃত হয়? [Intermediate]

ক) নেটওয়ার্ক ক্যাবল পরীক্ষা
খ) ইউজার ইন্টারফেস
গ) রিকোয়েস্ট-ভিত্তিক সার্ভিসের স্বাস্থ্য বোঝা — অনুরোধের হার, ত্রুটির হার ও সময়কাল
ঘ) ডেটাবেস ডিজাইন

৪৫. USE মেথড (Utilization, Saturation, Errors) মূলত কীসের জন্য? [Intermediate]

ক) ফ্রন্টএন্ড টেস্ট
খ) লগ বিশ্লেষণ
গ) প্রতিটি রিসোর্সের (CPU, মেমোরি, ডিস্ক, নেটওয়ার্ক) ব্যবহার, স্যাচুরেশন ও ত্রুটি বোঝা
ঘ) API ডিজাইন

৪৬. Alert fatigue কমাতে সবচেয়ে কার্যকর উপায় কোনটি? [Advanced]

ক) সব মেট্রিকে অ্যালার্ট বসানো
খ) অ্যালার্টের শব্দ বাড়ানো
গ) সব অ্যালার্ট SMS-এ পাঠানো
ঘ) শুধু ব্যবহারকারীকে প্রভাবিত করে এমন, কার্যকর (actionable) অ্যালার্ট রাখা এবং থ্রেশহোল্ড ও runbook যুক্ত করা

৪৭. একটি অ্যালার্ট বাজছে কিন্তু কোনো ব্যবহারকারী-প্রভাব নেই ও কোনো পদক্ষেপ স্পষ্ট নয়। কী করবেন? [Intermediate]

ক) অ্যালার্টটি উপেক্ষা করে রাখা
খ) অ্যালার্টটি টিউন বা সরিয়ে দেওয়া — non-actionable অ্যালার্ট শব্দদূষণ তৈরি করে ও আসল অ্যালার্ট মিস করায়
গ) সবাইকে ফোন করা
ঘ) সার্ভার রিস্টার্ট করা

৪৮. Prometheus-এ counter মেট্রিকের বৈশিষ্ট্য কী? [Beginner]

ক) মান কেবল বাড়ে (বা রিসেটে শূন্য হয়) — যেমন মোট রিকোয়েস্ট সংখ্যা
খ) মান বাড়ে-কমে
গ) শুধু লেটেন্সি মাপে
ঘ) লেবেল থাকতে পারে না

৪৯. Histogram কেন latency মাপতে ভালো? [Intermediate]

ক) গড় (average) সবসময় সত্য বলে
খ) এটি শুধু সর্বোচ্চ মান দেখায়
গ) এটি ডিস্ক বাঁচায়
ঘ) এটি বিতরণ (distribution) দেখায় — p50/p95/p99 বের করা যায়, গড় লুকিয়ে রাখা tail latency ধরা যায়

৫০. OpenTelemetry-এর মূল সুবিধা কী? [Beginner]

ক) ভেন্ডর-নিরপেক্ষভাবে trace, metric ও log সংগ্রহ ও পাঠানোর মান (standard)
খ) শুধু Grafana-র জন্য
গ) ডেটাবেস চালানো
ঘ) শুধু লগ সংগ্রহ করা

৫১. Distributed tracing-এ span কী? [Beginner]

ক) সার্ভারের নাম
খ) একটি রিকোয়েস্টের ভেতরে একটি একক কাজের সময়কালসহ রেকর্ড — একাধিক span মিলে একটি trace তৈরি হয়
গ) লগ ফাইলের আকার
ঘ) নেটওয়ার্ক ক্যাবল

৫২. লগ, মেট্রিক ও ট্রেস — তিনটি একসাথে থাকলে কী সুবিধা হয়? [Intermediate]

ক) স্টোরেজ খরচ কমে
খ) শুধু ড্যাশবোর্ড সুন্দর হয়
গ) কোনো ঘটনার কখন (metric), কোথায় (trace) ও কেন (log) — তিনটি প্রশ্নের উত্তর মেলে; একটিতে শুরু করে

অন্যটিতে গভীরে যাওয়া যায়
ঘ) অ্যালার্ট কমে যায়

৫৩. Incident management-এ প্রথম ৫ মিনিটে সবচেয়ে গুরুত্বপূর্ণ কাজ কোনটি? [Advanced]

ক) প্রভাব (impact) নির্ণয়, যোগাযোগ শুরু ও ব্যবহারকারীর ক্ষতি কমানো (mitigate) — রুট কজ পরে
খ) দোষী খোঁজা
গ) রুট কজ বিশ্লেষণ শেষ করা
ঘ) পোস্টমর্টেম লেখা

৫৪. Blameless postmortem কেন গুরুত্বপূর্ণ? [Intermediate]

ক) কাউকে দোষ না দিলে সবাই খুশি থাকে
খ) রিপোর্ট ছোট হয়
গ) দোষ না খুঁজে সিস্টেমের দুর্বলতা ও প্রক্রিয়াগত কারণ খুঁজে বের করা যায় — তাতে সত্য তথ্য বের হয় ও পুনরাবৃত্তি রোধ হয়
ঘ) মিটিং কম হয়

৫৫. SEV-1 ইনসিডেন্টের সংজ্ঞা কী? [Beginner]

ক) ছোটখাটো কসমেটিক বাগ
খ) ভবিষ্যতের উন্নতির অনুরোধ
গ) শুধু একটি ইউজারের সমস্যা
ঘ) গুরুতর সেবা ব্যাহত — বেশিরভাগ ব্যবহারকারী আক্রান্ত বা ডেটা ক্ষতির ঝুঁকি, অবিলম্বে সাড়া প্রয়োজন

৫৬. MTTR (Mean Time To Recover) কমাতে কোনটি সবচেয়ে কার্যকর? [Intermediate]

ক) বেশি অ্যালার্ট তৈরি করা
খ) রানবুক (runbook), অনুশীলিত DR/রোলব্যাক পরিকল্পনা ও স্পষ্ট ভূমিকা (Incident Commander, comms) থাকা
গ) মিটিং বাড়ানো
ঘ) লগ মুছে ফেলা

৫৭. RCA (Root Cause Analysis)-এ "5 Whys" কৌশলের উদ্দেশ্য কী? [Intermediate]

ক) পাঁচজনকে জিজ্ঞাসা করা
খ) বারবার "কেন" জিজ্ঞাসা করে লক্ষণের পেছনের গভীর কারণে পৌঁছানো
গ) পাঁচটি টুল ব্যবহার করা
ঘ) পাঁচ ঘণ্টা মিটিং করা

৫৮. একটি ডেপ্লয়ের পরপরই error rate বেড়ে গেছে। প্রথমে কী করবেন? [Advanced]

ক) নতুন ফিচার আরও ডেপ্লয় করা
খ) লগ মুছে ফেলা
গ) রুট কজ খুঁজতে ঘণ্টার পর ঘণ্টা অপেক্ষা করা
ঘ) দ্রুত রোলব্যাক করে সেবা স্থিতিশীল করা, তারপর শান্তভাবে কারণ বিশ্লেষণ করা

৫৯. Canary deployment-এর সুবিধা কী? [Intermediate]

ক) সব ইউজারকে একসাথে নতুন সংস্করণ দেওয়া
খ) ডেপ্লয় দ্রুত করা
গ) অল্প সংখ্যক ব্যবহারকারীর কাছে নতুন সংস্করণ পরীক্ষা করে ঝুঁকি সীমিত রাখা ও সমস্যা হলে সহজে ফেরা
ঘ) ডেটাবেস মুছে ফেলা

৬০. Blue-Green deployment কী? [Beginner]

ক) দুটি পরিবেশ (Blue = বর্তমান, Green = নতুন) রেখে ট্রাফিক সুইচ করা — রোলব্যাক মানে ট্রাফিক ফিরিয়ে দেওয়া
খ) দুটি ডেটাবেস একসাথে চালানো
গ) দুটি রং দিয়ে UI সাজানো
ঘ) দুটি সার্ভারে লোড ভাগ করা

৬১. Load testing ও stress testing-এর পার্থক্য কী? [Intermediate]

ক) দুটি একই জিনিস
খ) Stress test শুধু UI দেখে
গ) Load test প্রত্যাশিত লোডে আচরণ যাচাই করে; stress test সীমার বাইরে চাপ দিয়ে ভাঙন-বিন্দু (breaking point) ও পুনরুদ্ধার যাচাই করে
ঘ) Load test শুধু ডেটাবেসে হয়

৬২. ক্যাপাসিটি প্ল্যানিং-এ সবচেয়ে গুরুত্বপূর্ণ ইনপুট কোনটি? [Advanced]

ক) ঐতিহাসিক ট্রাফিক প্যাটার্ন, বৃদ্ধির হার, পিক/সিজনালিটি ও ব্যবসায়িক পরিকল্পনা — শুধু বর্তমান CPU ব্যবহার নয়
খ) সার্ভারের রং
গ) ডেভেলপারদের পছন্দ
ঘ) গত সপ্তাহের লগ

৬৩. Autoscaling-এ HPA (Horizontal Pod Autoscaler) কীসের ভিত্তিতে স্কেল করে? [Beginner]

ক) সময়ের ভিত্তিতে
খ) CPU/মেমোরি ব্যবহার বা কাস্টম মেট্রিকের ভিত্তিতে Pod-এর সংখ্যা বাড়ায়-কমায়
গ) ডেটাবেসের আকার দেখে
ঘ) ইউজারের নাম দেখে

৬৪. হঠাৎ ট্রাফিক ১০ গুণ বেড়ে গেছে, ডেটাবেস CPU ১০০%। প্রথমে কী করবেন? [Advanced]

ক) অ্যাপ সার্ভার বাড়ানো
খ) ক্যাশ বন্ধ করা
গ) আরও ফিচার ডেপ্লয় করা
ঘ) Read replica / ক্যাশ দিয়ে রিড লোড কমানো, ধীর কোয়েরি চিহ্নিত করা ও প্রয়োজনে rate limiting / load shedding দিয়ে ডেটাবেসকে রক্ষা করা

৬৫. RTO (Recovery Time Objective) কী? [Beginner]

ক) ব্যাকআপের আকার
খ) ডেটা কত পুরনো হতে পারে
গ) ব্যাকআপের খরচ
ঘ) বিপর্যয়ের পর সেবা পুনরুদ্ধারে সর্বোচ্চ গ্রহণযোগ্য সময়

৬৬. RPO (Recovery Point Objective) কী? [Beginner]

ক) পুনরুদ্ধারের সময়
খ) সর্বোচ্চ গ্রহণযোগ্য ডেটা ক্ষতি — যেমন ১৫ মিনিটের ডেটা হারানো যাবে
গ) সার্ভারের সংখ্যা
ঘ) ব্যাকআপের ফ্রিকোয়েন্সি নয়

৬৭. RTO ১ ঘণ্টা ও RPO ১৫ মিনিট হলে কোন কৌশল সবচেয়ে উপযুক্ত? [Intermediate]

ক) শুধু সাপ্তাহিক ব্যাকআপ
খ) শুধু ম্যানুয়াল ব্যাকআপ
গ) নিয়মিত স্বয়ংক্রিয় ব্যাকআপ + পয়েন্ট-ইন-টাইম রিকভারি + নিয়মিত restore পরীক্ষা
ঘ) ব্যাকআপ না নেওয়া

৬৮. ব্যাকআপ নেওয়া হলেও restore পরীক্ষা কেন জরুরি? [Intermediate]

ক) অপরীক্ষিত ব্যাকআপ প্রায়শই কাজ করে না — করাপ্ট, অসম্পূর্ণ বা ভুল কনফিগারেশনের কারণে; restore পরীক্ষাই প্রমাণ করে ব্যাকআপ সত্যিই কাজ করে
খ) ব্যাকআপ ফাইল সুন্দর দেখায়
গ) খরচ বাড়ে বলে
ঘ) নিয়ম মানে বলে

৬৯. Chaos engineering-এর উদ্দেশ্য কী? [Beginner]

ক) ইচ্ছাকৃতভাবে ছোট, নিয়ন্ত্রিত ব্যর্থতা ঘটিয়ে সিস্টেমের দুর্বলতা ও পুনরুদ্ধার ক্ষমতা আগেভাগে জানা
খ) ডেভেলপারদের ভয় দেখানো
গ) সার্ভার বন্ধ রাখা
ঘ) প্রোডাকশন ভেঙে ফেলা

৭০. Chaos experiment চালানোর আগে সবচেয়ে গুরুত্বপূর্ণ প্রস্তুতি কোনটি? [Advanced]

ক) সবাইকে না জানিয়ে চালানো
খ) প্রোডাকশনে সরাসরি বড় ব্যর্থতা ঘটানো
গ) হাইপোথিসিস, blast radius সীমিত করা, abort শর্ত, পর্যবেক্ষণ ও রোলব্যাক পরিকল্পনা ঠিক করা
ঘ) মনিটরিং বন্ধ করা

৭১. FinOps-এর মূল লক্ষ্য কী? [Beginner]

ক) ক্লাউড খরচ কমানোই একমাত্র লক্ষ্য
খ) সব রিসোর্স মুছে ফেলা
গ) বিল লুকিয়ে রাখা
ঘ) ক্লাউড খরচে স্বচ্ছতা, দায়বদ্ধতা ও মূল্য (value) — প্রয়োজনীয় খরচে সেরা ফল পাওয়া

৭২. ক্লাউড বিল হঠাৎ বেড়ে গেছে। প্রথমে কী খুঁজবেন? [Intermediate]

ক) বিলটি উপেক্ষা করা
খ) খরচের ভাঙন (cost breakdown) দেখে কোন সার্ভিস/রিসোর্স/ট্যাগে বৃদ্ধি, idle রিসোর্স ও অপ্রত্যাশিত ডেটা ট্রান্সফার খুঁজে বের করা

গ) সব সার্ভার বন্ধ করা
ঘ) বাজেট বাড়িয়ে দেওয়া

৭৩. Idle রিসোর্স কমাতে কোনটি সবচেয়ে কার্যকর? [Intermediate]

ক) সব রিসোর্স বড় করা
খ) রিসোর্স মুছে ফেলা যাবে না ভাবা
গ) ব্যবহারের মেট্রিক দেখে অব্যবহৃত/কম-ব্যবহৃত রিসোর্স right-size করা বা বন্ধ করা ও অটো-শাটডাউন নীতি করা
ঘ) বিল না দেখা

৭৪. Governance ও compliance-এ Policy as Code-এর সুবিধা কী? [Beginner]

ক) নীতি কোড হিসেবে সংস্করণ-নিয়ন্ত্রিত, পরীক্ষাযোগ্য ও স্বয়ংক্রিয়ভাবে প্রয়োগযোগ্য হয়
খ) নীতি কাগজে লেখা
গ) নীতি মুখে বলা
ঘ) ম্যানুয়াল চেক

৭৫. SOC 2 / ISO 27001-এর মতো ফ্রেমওয়ার্কে সবচেয়ে গুরুত্বপূর্ণ প্রমাণ কী? [Advanced]

ক) সুন্দর পলিসি ডকুমেন্ট
খ) বড় টিম
গ) দামি টুল
ঘ) নিয়ন্ত্রণ (control) আসলেই চলছে তার প্রমাণ — লগ, টিকিট, রিভিউ রেকর্ড ও ধারাবাহিক প্রয়োগের প্রমাণ

৭৬. Secrets management-এ HashiCorp Vault-এর সুবিধা কী? [Intermediate]

ক) সিক্রেট গিটে রাখা যায়
খ) কেন্দ্রীয়ভাবে সিক্রেট সংরক্ষণ, স্বল্পমেয়াদি (dynamic) সিক্রেট দেওয়া, অডিট লগ ও অ্যাক্সেস নিয়ন্ত্রণ
গ) পাসওয়ার্ড মনে রাখা
ঘ) সিক্রেট ইমেইলে পাঠানো

৭৭. TLS সার্টিফিকেট মেয়াদোত্তীর্ণ হলে কী হয়? [Beginner]

ক) সাইট দ্রুত হয়
খ) ব্রাউজার সতর্কবার্তা দেয় ও সংযোগ অবিশ্বস্ত দেখায়; স্বয়ংক্রিয় নবায়ন (auto-renewal) ও মেয়াদ-অ্যালার্ট জরুরি
গ) কিছুই হয় না
ঘ) ডেটা আরও নিরাপদ হয়

৭৮. WAF (Web Application Firewall) কী থেকে রক্ষা করে? [Intermediate]

ক) হার্ডওয়্যার চুরি
খ) DDoS সম্পূর্ণভাবে
গ) HTTP স্তরে সাধারণ আক্রমণ (SQLi, XSS প্যাটার্ন) ফিল্টার করে — তবে এটি নিরাপদ কোডের বিকল্প নয়
ঘ) পাসওয়ার্ড ফাঁস

৭৯. Rate limiting কেন দরকার? [Beginner]

ক) অপব্যবহার, ব্রুট-ফোর্স ও হঠাৎ লোড থেকে সেবাকে রক্ষা করা
খ) ইউজারকে বিরক্ত করা

গ) সাইট ধীর করা
ঘ) API ধীর করা

৮০. একটি API-তে ব্রুট-ফোর্স লগইন চেষ্টা দেখা যাচ্ছে। সবচেয়ে ভালো প্রতিক্রিয়া কোনটি? [Advanced]

ক) লগইন পেজ বন্ধ করা
খ) সব ইউজারের পাসওয়ার্ড বদলে দেওয়া
গ) কিছু না করা
ঘ) Rate limiting + অ্যাকাউন্ট লকআউট/ক্যাপচা + MFA উৎসাহিত করা এবং সন্দেহজনক IP পর্যবেক্ষণ করা

৮১. Log retention নীতি নির্ধারণের সময় কী বিবেচনা করবেন? [Intermediate]

ক) যত বেশি লগ তত ভালো
খ) লগ মুছে ফেলা
গ) লগ না রাখা
ঘ) আইনি/কমপ্লায়েন্স প্রয়োজন, ইনসিডেন্ট তদন্তের সময়কাল, স্টোরেজ খরচ ও সংবেদনশীল ডেটা মাস্কিং

৮২. কোনটি observability-র তিনটি স্তম্ভ (three pillars)? [Beginner]

ক) Code, test, deploy
খ) CPU, মেমোরি, ডিস্ক
গ) Logs, Metrics, Traces
ঘ) Alert, dashboard, report

৮৩. SLO-ভিত্তিক অ্যালার্ট (burn-rate alert) কেন থ্রেশহোল্ড অ্যালার্টের চেয়ে ভালো? [Advanced]

ক) এটি বেশি অ্যালার্ট দেয়
খ) এটি error budget কত দ্রুত খরচ হচ্ছে তা দেখায় — ব্যবহারকারীর অভিজ্ঞতার সাথে সরাসরি যুক্ত, তাই অপ্রয়োজনীয় অ্যালার্ট কমে
গ) এটি সস্তা
ঘ) এটি সহজ

৮৪. ৯৯.৯৯% SLO-র ৩০ দিনের error budget কত? [Intermediate]

ক) ৪ মিনিট ১৯ সেকেন্ড
খ) ৪৩ মিনিট
গ) ৭ ঘণ্টা
ঘ) ১ দিন

৮৫. ৯৯.৯% SLO-র ৩০ দিনের error budget কত? [Beginner]

ক) ৪৩ মিনিট ১২ সেকেন্ড
খ) ৪ মিনিট
গ) ৭ ঘণ্টা
ঘ) ১০ মিনিট

৮৬. একটি সার্ভিসের p95 latency বেড়েছে কিন্তু গড় (average) প্রায় একই আছে। এর অর্থ কী? [Advanced]

ক) কোনো সমস্যা নেই
খ) সব রিকোয়েস্ট ধীর হয়েছে
গ) মনিটরিং ভুল
ঘ) অল্প কিছু রিকোয়েস্ট (tail) অনেক ধীর হচ্ছে — গড় তা লুকিয়ে রাখে; tail latency তদন্ত দরকার

৮৭. Database connection pool শেষ হয়ে গেলে সাধারণ লক্ষণ কী? [Intermediate]

ক) CPU কমে যাওয়া
খ) ডিস্ক খালি হওয়া
গ) রিকোয়েস্ট আটকে যাওয়া / timeout বেড়ে যাওয়া, যদিও অ্যাপ সার্ভার সুস্থ
ঘ) লগ বন্ধ হওয়া

৮৮. Circuit breaker প্যাটার্নের কাজ কী? [Intermediate]

ক) সার্ভার বন্ধ করা
খ) ব্যর্থ ডিপেন্ডেন্সিতে বারবার রিকোয়েস্ট পাঠিয়ে তাকে আরও চাপ দেওয়া থেকে বিরত রাখা ও দ্রুত fail-fast করা
গ) নেটওয়ার্ক ক্যাবল কাটা
ঘ) ডেটাবেস মুছে ফেলা

৮৯. Retry করার সময় exponential backoff + jitter কেন ব্যবহার করা হয়? [Advanced]

ক) Retry দ্রুত করা
খ) একসাথে অনেক retry (thundering herd) এড়িয়ে ব্যর্থ সার্ভিসকে শ্বাস নেওয়ার সুযোগ দেওয়া
গ) লগ বাড়ানো
ঘ) কোড ছোট করা

৯০. Feature flag-এর নিরাপত্তা ও নির্ভরযোগ্যতায় সুবিধা কী? [Intermediate]

ক) কোড ডেপ্লয় না করে ঝুঁকিপূর্ণ ফিচার দ্রুত বন্ধ (kill switch) করা যায়
খ) ফিচার লুকানো
গ) কোড ডিলিট করা
ঘ) ডেপ্লয় না করা

৯১. Immutable infrastructure-এর অর্থ কী? [Intermediate]

ক) সার্ভার কখনো আপডেট না করা
খ) সার্ভার ম্যানুয়ালি ঠিক করা
গ) সার্ভারে প্যাচ দিয়ে বদল না করে নতুন ইমেজ/সার্ভার দিয়ে প্রতিস্থাপন করা
ঘ) সার্ভার বন্ধ রাখা

৯২. Infrastructure as Code (IaC) স্ক্যান করা কেন জরুরি? [Intermediate]

ক) ডেপ্লয় ধীর হয় বলে
খ) খরচ বাড়ে বলে
গ) কোড সুন্দর হয় বলে
ঘ) Terraform/Helm ফাইলে ভুল কনফিগারেশন (যেমন public বাকেট, খোলা পোর্ট) ডেপ্লয়ের আগেই ধরা যায়

৯৩. GitOps-এর মূল ধারণা কোনটি? [Beginner]

ক) গিটে সিক্রেট রাখা
খ) ম্যানুয়াল kubectl apply
গ) গিটকে সত্যের একমাত্র উৎস (single source of truth) ধরে স্বয়ংক্রিয়ভাবে ক্লাস্টারের অবস্থা গিটের সাথে মেলানো
ঘ) গিটহাব বন্ধ করা

৯৪. প্রোডাকশনে হটফিক্স দরকার, কিন্তু পূর্ণ CI পাইপলাইন চালাতে সময় লাগবে। সঠিক সিদ্ধান্ত কোনটি? [Advanced]

ক) পাইপলাইন পুরো বাইপাস করে সরাসরি প্রোডাকশনে পরিবর্তন করা
খ) হটফিক্স না করা
গ) টিমকে না জানানো
ঘ) ঝুঁকি নথিভুক্ত করে অনুমোদিত জরুরি পথে (emergency path) ন্যূনতম পরীক্ষাসহ ডেপ্লয় করা এবং পরে পূর্ণ পাইপলাইন দিয়ে যাচাই ও নথিভুক্ত করা

৯৫. Data breach হয়েছে সন্দেহ হলে প্রথম পদক্ষেপ কী? [Advanced]

ক) প্রভাব সীমিত করা (contain), প্রমাণ সংরক্ষণ করা, ইনসিডেন্ট প্রক্রিয়া শুরু করা ও আইনি/কমপ্লায়েন্স দলকে জানানো
খ) প্রমাণ মুছে ফেলা
গ) সবাইকে জানানো
ঘ) অপেক্ষা করা

৯৬. Least privilege লঙ্ঘনের উদাহরণ কোনটি? [Beginner]

ক) ডেভেলপারকে শুধু তার প্রজেক্টের read অ্যাক্সেস দেওয়া
খ) সব ডেভেলপারকে প্রোডাকশন ডেটাবেসে admin অ্যাক্সেস দেওয়া
গ) MFA চালু করা
ঘ) স্বল্পমেয়াদি টোকেন দেওয়া

৯৭. Container escape বলতে কী বোঝায়? [Intermediate]

ক) কনটেইনার থেকে বের হয়ে হোস্ট সিস্টেমে অ্যাক্সেস পাওয়া — যেমন privileged কনটেইনার বা ভুল কনফিগারেশনের মাধ্যমে
খ) ইমেজ ডাউনলোড করা
গ) Pod রিস্টার্ট হওয়া
ঘ) কনটেইনার মুছে ফেলা

৯৮. Admission controller-এর কাজ কী? [Intermediate]

ক) ইউজার লগইন করা
খ) Kubernetes-এ রিসোর্স তৈরি/পরিবর্তনের অনুরোধ নীতি অনুযায়ী যাচাই করে অনুমোদন বা প্রত্যাখ্যান করা
গ) লগ সংগ্রহ করা
ঘ) ডেটাবেস ব্যাকআপ

৯৯. একটি নতুন CVE-তে আপনার ব্যবহৃত লাইব্রেরির নাম আছে, কিন্তু আপনার কোড সেই vulnerable ফাংশনটি ব্যবহার করে না। কী করবেন? [Advanced]

ক) কিছুই না করা
খ) CVE নম্বরটি লুকানো
গ) সব সার্ভার বন্ধ করা

ঘ) Reachability/শোষণযোগ্যতা যাচাই করে ঝুঁকি নথিভুক্ত করা, তবুও আপডেটের পরিকল্পনা করা — কারণ ভবিষ্যতে ব্যবহার হতে পারে ও কমপ্লায়েন্সে ধরা পড়ে

১০০. SRE-তে toil (পুনরাবৃত্তিমূলক ম্যানুয়াল কাজ) কমানো কেন গুরুত্বপূর্ণ? [Intermediate]

ক) Toil মানে ভালো কাজ
খ) ম্যানুয়াল কাজ মজার নয় বলে
গ) Toil কমালে ইঞ্জিনিয়াররা উদ্ভাবন ও নির্ভরযোগ্যতার কাজে সময় পান; toil বাড়লে বার্নআউট ও ভুল বাড়ে
ঘ) Toil বাড়ানো উচিত

১০১. Multi-region DR-এ active-active বনাম active-passive — কখন active-passive যথেষ্ট? [Advanced]

ক) সবসময় active-active দরকার
খ) RTO খুব কম হলে
গ) খরচ কোনো বিষয় নয়
ঘ) RTO/RPO কিছুটা ঢিলা হলে ও খরচ নিয়ন্ত্রণে রাখতে চাইলে — passive অঞ্চল প্রস্তুত রেখে প্রয়োজনে ফেইলওভার করা

১০২. Load balancer-এর health check ভুল কনফিগার করা থাকলে কী হয়? [Intermediate]

ক) ট্রাফিক বাড়ে
খ) সাইট দ্রুত হয়
গ) অসুস্থ ইনস্ট্যান্সে ট্রাফিক যেতে থাকে বা সুস্থ ইনস্ট্যান্স সরিয়ে নেওয়া হয় — দুটোই বিপজ্জনক
ঘ) কিছুই হয় না

১০৩. কোনটি defence in depth-এর উদাহরণ? [Beginner]

ক) WAF + নিরাপদ কোড + least-privilege IAM + মনিটরিং — একাধিক স্তরে প্রতিরক্ষা, যাতে একটি স্তর ভাঙলেও অন্যটি রক্ষা করে
খ) শুধু ফায়ারওয়াল
গ) শুধু শক্ত পাসওয়ার্ড
ঘ) শুধু অ্যান্টিভাইরাস

১০৪. Security champion প্রোগ্রামের উদ্দেশ্য কী? [Intermediate]

ক) নিরাপত্তা দলকে বড় করা
খ) প্রতিটি ডেভ টিমে একজন নিরাপত্তা-সচেতন সদস্য রেখে নিরাপত্তা চর্চা দলে ছড়িয়ে দেওয়া
গ) পেনাল্টি দেওয়া
ঘ) অডিট করা

১০৫. DORA মেট্রিকের মধ্যে কোনটি পরিবর্তনের গতি মাপে? [Intermediate]

ক) Change failure rate
খ) Deployment frequency — কত ঘন ঘন সফলভাবে ডেপ্লয় হয়
গ) MTTR
ঘ) Lead time নয়

১০৬. Change failure rate বেশি হলে কী বোঝা যায়? [Advanced]

ক) ডেপ্লয় দ্রুত হচ্ছে
খ) টিম বড়
গ) কোড ভালো
ঘ) অনেক পরিবর্তনই ইনসিডেন্ট বা রোলব্যাক ঘটাচ্ছে — পরীক্ষা, রিভিউ বা ক্যানারি প্রক্রিয়ায় দুর্বলতা আছে

১০৭. কোনটি ভালো SLO-র বৈশিষ্ট্য? [Intermediate]

ক) ব্যবহারকারীর অভিজ্ঞতার সাথে যুক্ত, পরিমাপযোগ্য, অর্জনযোগ্য ও নির্দিষ্ট সময়সীমাসহ
খ) ১০০% হওয়া
গ) কখনো বদলানো যাবে না
ঘ) শুধু ইঞ্জিনিয়ারদের জন্য

১০৮. Incident-এর সময় যোগাযোগ (communication)-এ সবচেয়ে গুরুত্বপূর্ণ কী? [Intermediate]

ক) সব তথ্য গোপন রাখা
খ) দোষী খোঁজা
গ) নিয়মিত, সৎ ও সংক্ষিপ্ত আপডেট — প্রভাব, কী করা হচ্ছে ও পরের আপডেট কখন — একটি নির্দিষ্ট চ্যানেলে
ঘ) শুধু ম্যানেজারকে জানানো

১০৯. Post-incident review-এ সবচেয়ে মূল্যবান আউটপুট কোনটি? [Advanced]

ক) দোষীর নাম
খ) দীর্ঘ রিপোর্ট
গ) সুনির্দিষ্ট, দায়িত্বপ্রাপ্ত ও সময়সীমাসহ action items — যা সিস্টেম ও প্রক্রিয়া বদলায়
ঘ) মিটিংয়ের ছবি

১১০. কোনটি সঠিক backup কৌশল? [Beginner]

ক) 3-2-1 নিয়ম — তিন কপি, দুই ধরনের মাধ্যমে, একটি অফসাইট/অপরিবর্তনীয় (immutable) কপি — ও নিয়মিত restore পরীক্ষা
খ) এক কপি রাখা
গ) শুধু একই সার্ভারে রাখা
ঘ) ব্যাকআপ না নেওয়া

১১১. Kubernetes-এ liveness probe ও readiness probe-এর পার্থক্য কী? [Intermediate]

ক) দুটি একই
খ) Liveness শুধু লগ দেখে
গ) Readiness শুধু CPU দেখে
ঘ) Liveness বলে Pod বেঁচে আছে কি না (ব্যর্থ হলে রিস্টার্ট); readiness বলে Pod ট্রাফিক নিতে প্রস্তুত কি না (না হলে ট্রাফিক সরানো হয়, রিস্টার্ট নয়)

১১২. একটি সার্ভিসের error budget দ্রুত শেষ হচ্ছে কিন্তু ব্যবসা নতুন ফিচার চায়। SRE হিসেবে আপনি কী করবেন? [Advanced]

ক) চুপচাপ ফিচার রিলিজ করা
খ) তথ্য (burn rate, ব্যবহারকারী-প্রভাব, ঝুঁকি) দিয়ে ব্যবসার সাথে আলোচনা করে ফ্রিজ বা সীমিত রিলিজের সিদ্ধান্ত নেওয়া

গ) SLO গোপন করা
ঘ) মনিটরিং বন্ধ করা

১১৩. Supply-chain আক্রমণের উদাহরণ কোনটি? [Intermediate]

ক) ফিশিং ইমেইল
খ) ব্রুট-ফোর্স
গ) SQL Injection
ঘ) বিশ্বস্ত প্যাকেজ বা বিল্ড টুলে ক্ষতিকর কোড ঢুকিয়ে অনেক ভুক্তভোগীর কাছে পৌঁছে দেওয়া

১১৪. কোনটি নিরাপদ API ডিজাইনের নীতি? [Beginner]

ক) সব এন্ডপয়েন্টে প্রমাণীকরণ ও অনুমোদন, ইনপুট যাচাই, rate limiting ও সংবেদনশীল ডেটা ফাঁস না করা
খ) সব এন্ডপয়েন্ট খোলা রাখা
গ) API key URL-এ পাঠানো
ঘ) বিস্তারিত error stack trace ফেরত দেওয়া

১১৫. Observability বনাম monitoring-এর পার্থক্য কী? [Intermediate]

ক) দুটি একই
খ) Monitoring জানা সমস্যার জন্য আগে থেকে নির্ধারিত মেট্রিক দেখে; observability অজানা সমস্যার কারণ খুঁজতে সিস্টেমের ভেতরের অবস্থা জিজ্ঞাসা করার ক্ষমতা দেয়
গ) Monitoring বেশি দামি
ঘ) Observability শুধু লগ

১১৬. কোনটি chaos engineering-এর জন্য উপযুক্ত প্রথম পরীক্ষা? [Intermediate]

ক) প্রোডাকশন ডেটাবেস মুছে ফেলা
খ) পুরো অঞ্চল বন্ধ করা
গ) স্টেজিংয়ে একটি Pod বন্ধ করে সার্ভিস নিজে থেকে পুনরুদ্ধার হয় কি না দেখা — ছোট blast radius দিয়ে শুরু করা
ঘ) সব সার্ভার বন্ধ করা

১১৭. FinOps-এ tagging (ট্যাগিং) কেন গুরুত্বপূর্ণ? [Beginner]

ক) কোন টিম/প্রজেক্ট/পরিবেশের জন্য কত খরচ হচ্ছে তা জানা যায় — দায়বদ্ধতা ও অপ্টিমাইজেশনের ভিত্তি
খ) রিসোর্স সুন্দর দেখায়
গ) খরচ কমে
ঘ) বিল বড় হয়

১১৮. কোনটি shift-left security-র উদাহরণ? [Beginner]

ক) রিলিজের পরে পেন টেস্ট
খ) প্রোডাকশনে WAF
গ) IDE-তে লিন্টার/সিক্রেট স্ক্যান, PR-এ SAST/SCA ও IaC স্ক্যান — ডেভেলপমেন্টের শুরুতেই নিরাপত্তা ধরা
ঘ) অডিটের সময় নিরাপত্তা দেখা

১১৯. একটি ইনসিডেন্টে ডেটা ফাঁসের আশঙ্কা আছে। আপনার প্রথম অগ্রাধিকার কী? [Advanced]

ক) প্রেস রিলিজ লেখা
খ) প্রমাণ সংরক্ষণ ও প্রভাব সীমিত করে (contain) কারা, কী ডেটা ও কতটা — তা নির্ণয় করা এবং আইনি/নিরাপত্তা দলকে জানানো
গ) সব লগ মুছে ফেলা
ঘ) সোশ্যাল মিডিয়ায় পোস্ট করা

১২০. একজন ভালো DevSecOps / SRE ইঞ্জিনিয়ারের সবচেয়ে গুরুত্বপূর্ণ গুণ কোনটি? [Advanced]

ক) সব টুল জানা
খ) দ্রুত কোড লেখা
গ) বেশি ঘণ্টা কাজ করা
ঘ) ঝুঁকি, ট্রেড-অফ ও ব্যবহারকারী-প্রভাব বুঝে নিরাপত্তা, গতি ও নির্ভরযোগ্যতার ভারসাম্য রেখে সিদ্ধান্ত নেওয়া ও তা স্পষ্টভাবে যোগাযোগ করা

সংক্ষিপ্ত উত্তর • ৩০টি প্রশ্ন

সংক্ষিপ্ত প্রশ্ন — ৩০টি

প্রতিটি প্রশ্নের উত্তর ২–৪ বাক্যে লিখুন। সংজ্ঞার সাথে একটি উদাহরণ বা কেন এটি গুরুত্বপূর্ণ — তা যোগ করুন।

1. **Shift-left security কী?** [Beginner] — নিরাপত্তা পরীক্ষা ডেভেলপমেন্টের শুরুতে আনার ধারণাটি বুঝিয়ে লিখুন।
2. **Threat modeling-এর চারটি মৌলিক প্রশ্ন কী কী?** [Beginner]
3. **OWASP Top 10-এর মধ্যে Broken Access Control ও Cryptographic Failures-এর পার্থক্য উদাহরণসহ লিখুন।** [Intermediate]
4. **SAST, DAST ও SCA — তিনটি কখন ব্যবহার করবেন?** [Intermediate]
5. **SBOM কেন Log4Shell-এর মতো ঘটনায় কাজে আসে?** [Intermediate]
6. **SLSA Level 3-এ পৌঁছাতে একটি বিল্ড পাইপলাইনে কী কী থাকা দরকার?** [Advanced]
7. **কনটেইনার ইমেজে distroless বেস ব্যবহারের সুবিধা ও সীমা লিখুন।** [Intermediate]
8. **Kubernetes Pod Security Standards-এর তিনটি স্তর (Privileged, Baseline, Restricted) ব্যাখ্যা করুন।** [Intermediate]
9. **OPA-তে Policy as Code লেখার সুবিধা কী? একটি উদাহরণ দিন।** [Intermediate]
10. **CVSS Base, Temporal ও Environmental স্কোরের পার্থক্য লিখুন।** [Advanced]
11. **ঝুঁকি-ভিত্তিক vulnerability prioritization বলতে কী বোঝায়? একটি উদাহরণ দিন।** [Advanced]
12. **Cloud misconfiguration-এর তিনটি সাধারণ উদাহরণ ও প্রতিকার লিখুন।** [Intermediate]
13. **Secrets rotation বলতে কী বোঝায় এবং কেন এটি স্বয়ংক্রিয় হওয়া উচিত?** [Intermediate]
14. **Secure CI/CD পাইপলাইনের পাঁচটি অপরিহার্য নিয়ন্ত্রণ (control) লিখুন।** [Intermediate]
15. **SLI, SLO ও SLA — তিনটির সংজ্ঞা ও সম্পর্ক একটি উদাহরণ দিয়ে বুঝিয়ে লিখুন।** [Beginner]
16. **Error budget কীভাবে গণনা করা হয়? ৯৯.৯% SLO-র মাসিক বাজেট হিসাব করে দেখান।** [Intermediate]
17. **Golden Signals ও RED মেথডের মধ্যে সম্পর্ক লিখুন।** [Intermediate]
18. **Actionable alert-এর তিনটি বৈশিষ্ট্য লিখুন।** [Intermediate]
19. **Distributed tracing-এ trace ID কীভাবে একটি রিকোয়েস্টকে অনুসরণ করে?** [Intermediate]
20. **Blameless postmortem কী এবং কেন এটি দোষারোপের সংস্কৃতির চেয়ে ভালো?** [Beginner]
21. **Incident Commander-এর দায়িত্ব কী কী?** [Intermediate]
22. **5 Whys দিয়ে একটি ডেপ্লয়-পরবর্তী বিভ্রাটের রুট কজ বিশ্লেষণের উদাহরণ দিন।** [Intermediate]
23. **Canary, Blue-Green ও Rolling deployment-এর মধ্যে কখন কোনটি বেছে নেবেন?** [Advanced]
24. **Load test ও stress test-এর ফলাফল থেকে ক্যাপাসিটি সিদ্ধান্ত কীভাবে নেবেন?** [Advanced]
25. **RTO ও RPO-র পার্থক্য একটি ব্যাংকিং সেবার উদাহরণ দিয়ে বুঝিয়ে লিখুন।** [Intermediate]
26. **Chaos engineering-এর একটি নিরাপদ প্রথম পরীক্ষা ডিজাইন করুন (hypothesis, blast radius, abort শর্তসহ)।** [Advanced]
27. **FinOps-এ cost allocation-এ tagging-এর ভূমিকা লিখুন।** [Beginner]
28. **Toil কী? একটি toil কমানোর অটোমেশন উদাহরণ দিন।** [Intermediate]
29. **Defence in depth বলতে কী বোঝায়? একটি ওয়েব সেবার জন্য চারটি স্তর লিখুন।** [Beginner]

30. **DevSecOps ও SRE একসাথে কীভাবে একটি নির্ভরযোগ্য ও নিরাপদ সেবা তৈরি করে — তিন বাক্যে সারাংশ লিখুন।** [Advanced]

> **উত্তর লেখার টিপস:** সংজ্ঞা দিয়ে শুরু করুন, তারপর একটি ছোট উদাহরণ দিন, শেষে কেন গুরুত্বপূর্ণ বা কখন ব্যবহার করবেন না — তা যোগ করুন। শুধু বইয়ের লাইন কপি করলে পূর্ণ নম্বর মিলবে না।

সমস্যা সমাধান • ২০টি পরিস্থিতি

ট্রাবলশুটিং পরিস্থিতি — ২০টি

প্রতিটি পরিস্থিতি পড়ুন, তারপর নির্ণয় করুন: (ক) সম্ভাব্য কারণ কী, (খ) প্রথমে কী যাচাই করবেন, (গ) তাৎক্ষণিক প্রশমন (mitigation) কী, (ঘ) দীর্ঘমেয়াদি প্রতিকার কী। উত্তরে যুক্তি ও ট্রেড-অফ লিখুন।

1. **হঠাৎ login ব্যর্থতা বেড়ে গেছে।** শুক্রবার রাত ১১টায় login API-র error rate ০.৫% থেকে ১৮% হয়েছে, কিন্তু সার্ভার CPU ও মেমোরি স্বাভাবিক। গতকাল ডেপ্লয়ে পাসওয়ার্ড হ্যাশিং লাইব্রেরি আপডেট হয়েছিল। আপনি কী নির্ণয় করবেন, কীভাবে যাচাই করবেন ও প্রথম পদক্ষেপ কী হবে?
2. **CI-তে Trivy critical CVE ধরেছে, রিলিজ আজ।** পেমেন্ট সার্ভিসের ইমেজে একটি critical CVE পাওয়া গেছে, কিন্তু রিলিজ ব্যবসার জন্য জরুরি। CVE-টি একটি ব্যবহৃত না-হওয়া লাইব্রেরি ফাংশনে। আপনি কী সিদ্ধান্ত নেবেন — ব্লক করবেন, নাকি ঝুঁকি গ্রহণ করে এগোবেন? শর্ত ও নথিভুক্তি কী হবে?
3. **Kubernetes Pod বারবার CrashLoopBackOff-এ যাচ্ছে।** নতুন ইমেজ ডেপ্লয়ের পর একটি Pod চালু হয়েই থেমে যাচ্ছে, লগে `permission denied` দেখা যাচ্ছে। securityContext-এ সম্প্রতি `runAsNonRoot` যোগ করা হয়েছে। কারণ কী হতে পারে ও কীভাবে ঠিক করবেন?
4. **S3 বাকেট public — গ্রাহকের ফাইল দেখা যাচ্ছে।** একজন বাইরের ব্যক্তি জানালেন আপনার একটি বাকেটের ফাইল ইন্টারনেটে খোলা যাচ্ছে। আপনি প্রথম ৩০ মিনিটে কী করবেন, কীভাবে প্রভাব যাচাই করবেন ও পুনরাবৃত্তি কীভাবে রোধ করবেন?
5. **Prometheus-এ অ্যালার্ট বন্যা (alert storm)।** একটি ডেটাবেস ধীর হওয়ায় ৪০টি অ্যালার্ট একসাথে বাজছে — সব টিম একই ঘটনা দেখছে, কিন্তু কেউ জানে না আসল কারণ কী। আপনি কীভাবে অ্যালার্টগুলোকে একত্র করবেন ও মূল কারণে পৌঁছাবেন?
6. **p99 latency হঠাৎ দ্বিগুণ, p50 স্বাভাবিক।** চেকআউট API-র p99 ৪০০ms থেকে ৯০০ms হয়েছে, কিন্তু গড় ও p50 প্রায় একই আছে। ডেটাবেসে একটি নতুন ইনডেক্স যোগ করা হয়েছিল। আপনি কোন দিকে তদন্ত করবেন?
7. **ডেটাবেস কানেকশন শেষ (connection exhaustion)।** রাতের পিক আওয়ারে অ্যাপ `too many connections` ত্রুটি দিচ্ছে, কিন্তু ডেটাবেস CPU ৩০%। অ্যাপে সম্প্রতি নতুন মাইক্রোসার্ভিস যোগ হয়েছে। কারণ ও প্রতিকার কী?
8. **সিক্রেট গিটহাবে ফাঁস হয়েছে।** একজন ডেভেলপার ভুল করে প্রোডাকশন API key পাবলিক রিপোজিটরিতে পুশ করেছেন; কীটি ২ ঘণ্টা ধরে দৃশ্যমান ছিল। আপনার incident response পরিকল্পনা কী হবে?
9. **Canary ডেপ্লয়ে ৫% ট্রাফিকে error বেড়েছে।** নতুন সংস্করণে ৫% ব্যবহারকারীর কাছে error rate ৩ গুণ বেড়েছে, কিন্তু বাকি ৯৫% স্বাভাবিক। আপনি রোলব্যাক করবেন, নাকি তদন্ত চালিয়ে যাবেন? সিদ্ধান্তের মাপকাঠি কী?
10. **ক্লাউড বিল গত মাসের দ্বিগুণ।** ফাইন্যান্স টিম জানাল বিল দ্বিগুণ হয়েছে, কিন্তু ট্রাফিক বাড়েনি। কোন সার্ভিসে বৃদ্ধি তা খুঁজতে আপনি কোন ধাপে এগোবেন ও কী কী পরীক্ষা করবেন?
11. **সার্টিফিকেট মেয়াদোত্তীর্ণ — সাইটে সতর্কবার্তা।** সকালে গ্রাহকরা জানালেন সাইটে "Your connection is not private" দেখাচ্ছে। সার্টিফিকেট গতকাল মেয়াদোত্তীর্ণ হয়েছে, auto-renewal ব্যর্থ হয়েছিল। তাৎক্ষণিক ও দীর্ঘমেয়াদি পদক্ষেপ কী?
12. **একটি Pod অন্য Namespace-এর ডেটাবেসে পৌঁছে যাচ্ছে।** NetworkPolicy থাকার কথা থাকলেও একটি টেস্ট Pod প্রোডাকশন ডেটাবেসে সংযোগ করতে পারছে। আপনি কীভাবে যাচাই করবেন নীতিটি আসলেই প্রয়োগ হচ্ছে কি না?
13. **Error budget শেষ — ব্যবসা ফিচার চায়।** এই মাসে error budget প্রায় শেষ, কিন্তু প্রোডাক্ট টিম একটি বড় ফিচার রিলিজ করতে চায়। SRE হিসেবে আপনি তথ্য দিয়ে কীভাবে আলোচনা করবেন ও কী বিকল্প প্রস্তাব দেবেন?

14. **Log-এ সংবেদনশীল ডেটা (PII) পাওয়া গেছে।** অ্যাপ লগে গ্রাহকের জাতীয় পরিচয়পত্র নম্বর ও ফোন নম্বর দেখা যাচ্ছে, লগগুলো একটি থার্ড-পার্টি টুলে পাঠানো হয়। আপনি কী করবেন — লগ বন্ধ করবেন, মাস্ক করবেন, নাকি অন্য কিছু?
15. **ডেটাবেস ফেইলওভার পরীক্ষা ব্যর্থ হয়েছে।** DR ড্রিলে দেখা গেল প্রাইমারি ডেটাবেস বন্ধ করলে রেপ্লিকা প্রমোট হতে ২৫ মিনিট লাগে, কিন্তু RTO ১৫ মিনিট। আপনি কোন দিকে তদন্ত করবেন ও কী বদলাবেন?
16. **GitHub Actions-এ PR থেকে সিক্রেট ফাঁসের চেষ্টা।** একটি বাইরের কন্ট্রিবিউটরের PR-এ সন্দেহজনক কোড আছে যা পরিবেশ ভেরিয়েবল প্রিন্ট করার চেষ্টা করে। ওয়ার্কফ্লোটি PR-এ সিক্রেট ব্যবহার করে। আপনি কী করবেন?
17. **সার্ভিস ধীর — কিন্তু কোন সার্ভিস দায়ী জানা যাচ্ছে না।** একটি রিকোয়েস্ট ৫টি মাইক্রোসার্ভিস পেরিয়ে যায়; মোট latency ৩ সেকেন্ড, কিন্তু প্রতিটি সার্ভিসের ড্যাশবোর্ড স্বাভাবিক দেখাচ্ছে। Distributed tracing দিয়ে আপনি কীভাবে দায়ী span খুঁজবেন?
18. **ব্যাকআপ আছে, কিন্তু restore হচ্ছে না।** বছরের প্রথম restore পরীক্ষায় দেখা গেল ব্যাকআপ ফাইল করাপ্ট। শেষ সফল restore পরীক্ষা হয়েছিল ৮ মাস আগে। আপনার তাৎক্ষণিক ও প্রক্রিয়াগত প্রতিকার কী হবে?
19. **Rate limiting-এর কারণে বৈধ ব্যবহারকারী ব্লক হচ্ছে।** একটি জনপ্রিয় অফিসের সব ব্যবহারকারী একই NAT IP থেকে আসে; rate limit-এর কারণে তারা সবাই ব্লক হচ্ছে। আপনি কীভাবে বৈধ ট্রাফিক ও অপব্যবহারের মধ্যে ভারসাম্য করবেন?
20. **একটি নির্ভরতা (dependency) আপডেটে ব্রেকিং চেঞ্জ।** সকালে Dependabot একটি লাইব্রেরি আপডেট করেছে; বিকেলে প্রোডাকশনে একটি বিরল এন্ডপয়েন্টে ৫০০ ত্রুটি দেখা দিচ্ছে। আপনার রোলব্যাক, পিনিং ও ভবিষ্যৎ প্রতিরোধ পরিকল্পনা কী হবে?

প্রোডাকশন ইনসিডেন্ট • ১০টি কেস স্টাডি

প্রোডাকশন ইনসিডেন্ট কেস স্টাডি — ১০টি

প্রতিটি কেস পড়ুন, তারপর বিশ্লেষণ করুন: কী ঘটেছিল, রুট কজ কী ছিল, কোন নিয়ন্ত্রণ (control) ব্যর্থ হয়েছিল, কী শেখা যায় এবং কোন concrete action items নেওয়া উচিত।

১. কেস ১ — ডেপ্লয়ের ১০ মিনিট পর পেমেন্ট বন্ধ

একটি ই-কমার্স সাইটে শুক্রবার বিকেলে পেমেন্ট সার্ভিসের নতুন সংস্করণ ডেপ্লয় করা হয়। ১০ মিনিট পর গ্রাহকরা জানান পেমেন্ট ব্যর্থ হচ্ছে। তদন্তে দেখা যায় নতুন কোডে পেমেন্ট গেটওয়ের API key-র পরিবেশ ভেরিয়েবলের নাম বদলে গেছে (`PAYMENT_KEY` থেকে `PAYMENT_API_KEY`), কিন্তু ডেপ্লয় কনফিগে পুরনো নামই রয়ে গেছে। রোলব্যাক করতে ৪০ মিনিট লেগেছে, কারণ রোলব্যাক প্রক্রিয়াটি ম্যানুয়াল ছিল।

বিশ্লেষণের প্রশ্ন: (ক) এই ব্যর্থতা কোন ধাপে ধরা যেত? (খ) রোলব্যাক কেন এত ধীর হলো? (গ) কনফিগ পরিবর্তনের জন্য কোন নিয়ন্ত্রণ যোগ করবেন?

২. কেস ২ — ভুল S3 নীতি: ২ লাখ গ্রাহকের ডেটা প্রকাশ্যে

একটি স্টার্টআপের ডেভেলপার পরীক্ষার জন্য একটি S3 বাকেট public করেন, পরে ভুলে যান। তিন সপ্তাহ পর একজন নিরাপত্তা গবেষক জানান বাকেটে গ্রাহকের ইনভয়েস ও ঠিকানা খোলা আছে। কোম্পানির কোনো CSPM অ্যালার্ট ছিল না, বাকেটের অ্যাক্সেস লগও চালু ছিল না।

বিশ্লেষণের প্রশ্ন: (ক) কোন কোন নিয়ন্ত্রণ একসাথে ব্যর্থ হয়েছে? (খ) প্রভাব কীভাবে নির্ণয় করবেন, যখন অ্যাক্সেস লগ নেই? (গ) প্রতিরোধে কোন তিনটি স্বয়ংক্রিয় নিয়ন্ত্রণ সবচেয়ে কার্যকর হবে?

৩. কেস ৩ — Log4Shell-এর রাতে SBOM-এর মূল্য

Log4Shell (CVE-2021-44228) প্রকাশের রাতে একটি মাঝারি কোম্পানির SRE টিম জানতে পারে না তাদের কোন সার্ভিসে vulnerable Log4j সংস্করণ আছে, কারণ কোনো SBOM ছিল না। তারা প্রতিটি রিপোজিটরি হাতে খুঁজতে শুরু করে; প্যাচ করতে ৫ দিন লেগে যায়। প্রতিযোগী একটি কোম্পানি SBOM দিয়ে ৩ ঘণ্টায় তাদের আক্রান্ত সার্ভিস চিহ্নিত করে।

বিশ্লেষণের প্রশ্ন: (ক) SBOM ছাড়া কী কী ঝুঁকি তৈরি হয়? (খ) জরুরি প্যাচিং-এর জন্য কোন প্রক্রিয়া আগেভাগে থাকা উচিত ছিল? (গ) এই কেস থেকে আপনার পাইপলাইনে কী বদলাবেন?

৪. কেস ৪ — ভুল অ্যালার্ট: সবাই ঘুমিয়েছিল, সার্ভিস বন্ধ ছিল

একটি SaaS সার্ভিসের ডেটাবেস ডিস্ক ৯৮% পূর্ণ হয়ে সেবা বন্ধ হয়ে যায় ভোর ৩টায়। ডিস্ক-ব্যবহারের অ্যালার্ট ছিল, কিন্তু থ্রেশহোল্ড ছিল ৯৫% — আর অ্যালার্টটি শুধু ইমেইলে যেত, যা রাতে কেউ দেখেনি। আগের সপ্তাহে একই অ্যালার্ট তিনবার বেজেছিল কিন্তু কেউ পদক্ষেপ নেয়নি (alert fatigue)।

বিশ্লেষণের প্রশ্ন: (ক) অ্যালার্ট ডিজাইনে কী ভুল ছিল? (খ) ডিস্ক পূর্ণ হওয়ার পূর্বাভাস (forecast) কীভাবে দেওয়া যেত? (গ) On-call প্রক্রিয়ায় কী বদলাবেন?

৫. কেস ৫ — DNS বিভ্রাটে পুরো সাইট অদৃশ্য

একটি কোম্পানি DNS রেকর্ড পরিবর্তন করার সময় TTL না দেখে ভুল IP বসায়। পরিবর্তনটি দ্রুত ছড়িয়ে পড়ে, সাইট ২ ঘণ্টা ধরে খোলা যায়নি। রোলব্যাকের পরও কিছু ব্যবহারকারী পুরনো ক্যাশের কারণে আরও ১ ঘণ্টা সমস্যায় ছিলেন। কোনো runbook ছিল না।

বিশ্লেষণের প্রশ্ন: (ক) DNS পরিবর্তনের নিরাপদ প্রক্রিয়া কেমন হওয়া উচিত? (খ) TTL-এর ভূমিকা কী? (গ) যোগাযোগ পরিকল্পনায় কী থাকা উচিত ছিল?

৬. কেস ৬ — ক্যাপাসিটি অনুমান ভুল: ফ্ল্যাশ সেলে সাইট ধস

একটি ফ্যাশন সাইট ফ্ল্যাশ সেলের দিন ৫ গুণ ট্রাফিক পায়; লোড টেস্ট করা হয়েছিল স্বাভাবিক ট্রাফিকের ২ গুণ ধরে। ডেটাবেস কানেকশন শেষ হয়ে যায়, চেকআউট বন্ধ হয়ে যায়। ব্যবসার ক্ষতি হয় বড় অঙ্কের। পরে দেখা যায় ক্যাশিং বন্ধ ছিল, কারণ একটি কনফিগ পরিবর্তন ক্যাশ অকার্যকর করে দিয়েছিল।

বিশ্লেষণের প্রশ্ন: (ক) ক্যাপাসিটি প্ল্যানিং-এ কী ভুল হয়েছিল? (খ) ক্যাশ বন্ধ থাকা কেন ধরা পড়েনি? (গ) পরের ফ্ল্যাশ সেলের আগে কোন তিনটি পরীক্ষা করবেন?

৭. কেস ৭ — সাপ্লাই চেইন: বিশ্বস্ত প্যাকেজে ক্ষতিকর কোড

একটি জনপ্রিয় NPM প্যাকেজের নতুন সংস্করণে ক্ষতিকর কোড ঢুকে যায়; যারা `latest` ব্যবহার করত, তারা সবাই আক্রান্ত হয়। একটি কোম্পানি পিন করা সংস্করণ ও লকফাইল ব্যবহার করত বলে আক্রান্ত হয়নি; অন্য কোম্পানিটি আক্রান্ত হয়ে ডেটা ফাঁসের শিকার হয়।

বিশ্লেষণের প্রশ্ন: (ক) পিন করা সংস্করণ কেন রক্ষা করল? (খ) SBOM ও স্বাক্ষর যাচাই কীভাবে সাহায্য করত? (গ) নতুন প্যাকেজ সংস্করণ গ্রহণের নিরাপদ প্রক্রিয়া কেমন হবে?

৮. কেস ৮ — ভুল রোলব্যাক: ডেটা মাইগ্রেশন ফেরানো যায়নি

একটি রিলিজে ডেটাবেস স্কিমা বদল (কলাম মুছে ফেলা) করা হয়। রিলিজে বাগ ধরা পড়লে টিম কোড রোলব্যাক করে, কিন্তু ডেটাবেস মাইগ্রেশন ফেরানো যায়নি — কলামটি মুছে যাওয়ায় পুরনো কোডও ভেঙে যায়। সেবা ৩ ঘণ্টা বন্ধ থাকে, ব্যাকআপ থেকে পুনরুদ্ধার করতে হয়।

বিশ্লেষণের প্রশ্ন: (ক) Expand-contract প্যাটার্ন কীভাবে এই বিপদ এড়াত? (খ) ডেটাবেস পরিবর্তনের জন্য রোলব্যাক পরিকল্পনায় কী থাকা উচিত? (গ) মাইগ্রেশন পরীক্ষার প্রক্রিয়া কেমন হবে?

৯. কেস ৯ — Blameless postmortem যা সংস্কৃতি বদলে দিল

একটি জুনিয়র ইঞ্জিনিয়ার ভুল করে প্রোডাকশন ডেটাবেসে একটি delete কোয়েরি চালান। আগের সংস্কৃতিতে তাকে দোষারোপ করা হতো; এবার টিম blameless postmortem করে দেখে — প্রোডাকশন অ্যাক্সেস খুব সহজ ছিল, কোনো অনুমোদন (approval) লাগত না, ব্যাকআপ restore পরীক্ষা করা হয়নি। ফলে তারা অ্যাক্সেস নিয়ন্ত্রণ, অনুমোদন ও নিয়মিত restore ড্রিল চালু করে। পরের বছর একই ধরনের ভুল আর ইনসিডেন্ট হয়নি।

বিশ্লেষণের প্রশ্ন: (ক) এই কেসে সিস্টেমের দুর্বলতাগুলো কী কী? (খ) Blameless পদ্ধতি কীভাবে সত্য তথ্য বের করতে সাহায্য করল? (গ) আপনি আপনার টিমে এই সংস্কৃতি কীভাবে আনবেন?

১০. কেস ১০ — Chaos experiment-এ ধরা পড়া একক ব্যর্থতা-বিন্দু

একটি টিম স্টেজিংয়ে একটি availability zone বন্ধ করার পরীক্ষা চালায়। দেখা যায় তাদের সার্ভিসের সব Pod আসলে একই zone-এ চলছিল, কারণ affinity নিয়ম ভুল ছিল। প্রোডাকশনেও একই ভুল ছিল — একটি zone বিভ্রাটেই পুরো সেবা বন্ধ হয়ে যেত। পরীক্ষাটি প্রোডাকশনে যাওয়ার আগেই সমস্যাটি ধরা পড়ে।

বিশ্লেষণের প্রশ্ন: (ক) এই পরীক্ষাটি কেন মূল্যবান ছিল? (খ) Topology spread ও affinity যাচাইয়ের জন্য কোন স্বয়ংক্রিয় পরীক্ষা যোগ করবেন? (গ) Chaos experiment-এর blast radius সীমিত রাখার উপায় কী?

সামগ্রিক মূল্যায়ন • ১০০ নম্বর • ~৩ ঘণ্টা

ফাইনাল পরীক্ষা — ব্যবহারিক মূল্যায়ন (১০০ নম্বর)

সময়: ৩ ঘণ্টা | **মোট নম্বর:** ১০০ | **পাস নম্বর:** ৬০ (প্রতিটি অংশে ন্যূনতম ৪০% পেতে হবে) | **ধরন:** লিখিত + ব্যবহারিক (ল্যাব)

১. নির্দেশনা

- সব অংশের উত্তর দিতে হবে। কোনো অংশ ফাঁকা রাখলে সেই অংশে শূন্য ধরা হবে।
- ব্যবহারিক অংশ শুধু **অনুমোদিত আইসোলেটেড ল্যাব পরিবেশে** করবেন। কোনো বাস্তব প্রোডাকশন সিস্টেম, অন্যের অ্যাকাউন্ট বা নেটওয়ার্কে পরীক্ষা করবেন না।
- কোনো বাস্তব সিক্রেট ব্যবহার করবেন না — শুধু `CHANGE-ME-...` প্লেসহোল্ডার।
- উত্তরে যুক্তি, ট্রেড-অফ ও অনুমান (assumption) স্পষ্ট করে লিখুন। শুধু সঠিক উত্তর নয়, সিদ্ধান্তের কারণও মূল্যায়ন করা হবে।
- ক্যালকুলেটর ও বইয়ের রেফারেন্স অংশ (গ্লসারি, চেকলিস্ট, কমান্ড রেফারেন্স) ব্যবহার করা যাবে; ইন্টারনেট ব্যবহারের নিয়ম পরীক্ষকের নির্দেশ অনুযায়ী।

২. অংশ ক — বহুনির্বাচনী (২০ নম্বর, ২০টি প্রশ্ন × ১)

এই সহায়িকার MCQ অংশ থেকে ২০টি প্রশ্ন দেওয়া হবে (সব অধ্যায় থেকে)। প্রতিটি সঠিক উত্তরে ১ নম্বর; ভুল উত্তরে নম্বর কাটা হবে না। সময়: ~২০ মিনিট।

৩. অংশ খ — সংক্ষিপ্ত উত্তর (২০ নম্বর, ৫টি প্রশ্ন × ৪)

নিচের ধরনের ৫টি প্রশ্নের উত্তর ৩–৫ বাক্যে লিখুন (পরীক্ষায় নির্দিষ্ট প্রশ্ন দেওয়া হবে):

- SLI/SLO/error budget-এর একটি হিসাবসহ ব্যাখ্যা।
- SAST বনাম DAST বনাম SCA — কখন কোনটি।
- Zero Trust-এর একটি বাস্তব প্রয়োগ।
- RTO/RPO দিয়ে একটি DR পরিকল্পনার রূপরেখা।
- FinOps-এ একটি খরচ-অপ্টিমাইজেশন সিদ্ধান্তের যুক্তি।

৪. অংশ গ — পরিস্থিতি বিশ্লেষণ (৩০ নম্বর, ৩টি পরিস্থিতি × ১০)

তিনটি পরিস্থিতি দেওয়া হবে (এই সহায়িকার scenarios অংশ থেকে অভিযোজিত)। প্রতিটিতে লিখতে হবে: সম্ভাব্য কারণ, যাচাইয়ের ধাপ, তাৎক্ষণিক প্রশমন ও দীর্ঘমেয়াদি প্রতিকার। প্রতিটি উপাদানে নম্বর ভাগ করা আছে: কারণ নির্ণয় ৩, যাচাই ২, প্রশমন ২, দীর্ঘমেয়াদি প্রতিকার ৩।

৫. অংশ ঘ — ব্যবহারিক ল্যাব (৩০ নম্বর)

ল্যাবে (আইসোলেটেড পরিবেশে) নিচের কাজগুলো করুন ও আউটপুট/স্ক্রিনশটসহ সংক্ষিপ্ত প্রতিবেদন জমা দিন:

কাজ	নম্বর
১. একটি নমুনা অ্যাপে Trivy/Grype দিয়ে ইমেজ স্ক্যান করে critical/high CVE-র তালিকা ও প্রতিকার পরিকল্পনা লেখা	৮
২. একটি Kubernetes manifest-এ Pod Security, resource limit ও NetworkPolicy যোগ করে Conftest/OPA দিয়ে যাচাই	৮
৩. Prometheus query (PromQL) দিয়ে error rate ও p95 latency বের করা এবং একটি burn-rate অ্যালার্ট নিয়ম লেখা	৭
৪. ৯৯.৯% ও ৯৯.৯৯% SLO-র ৩০ দিনের error budget হিসাব ও একটি রোলব্যাক সিদ্ধান্তের যুক্তি লেখা	৭

৬. পাসের মানদণ্ড

- **মোট ৬০ বা তার বেশি** এবং প্রতিটি অংশে ন্যূনতম ৪০%।
- **Distinction (৮০+):** পরিস্থিতি ও ল্যাবে ট্রেড-অফ ও ঝুঁকি স্পষ্টভাবে ব্যাখ্যা করা হয়েছে।
- কোনো অংশে অনুমোদনহীন পরীক্ষা বা বাস্তব সিক্রেট ব্যবহার ধরা পড়লে পরীক্ষা বাতিল হবে।
- ফেল করলে দুর্বল অংশ চিহ্নিত করে পুনরায় প্রস্তুতি নিন — MCQ উত্তরমালার ব্যাখ্যা ও সংশ্লিষ্ট অধ্যায় আবার পড়ুন।

পরীক্ষকের জন্য নোট: অংশ গ ও ঘ-তে একাধিক গ্রহণযোগ্য উত্তর থাকতে পারে। মূল্যায়নে যুক্তির স্পষ্টতা, ঝুঁকি-সচেতনতা ও ব্যবহারকারী-প্রভাবের বিবেচনাকে অগ্রাধিকার দিন — শুধু কীওয়ার্ড মেলানোকে নয়।

উত্তরমালা • MCQ ১–৬০

উত্তর নির্দেশিকা — MCQ ১–৬০

প্রতিটি উত্তরে সঠিক অক্ষর ও সংক্ষিপ্ত ব্যাখ্যা দেওয়া আছে। ব্যাখ্যাটি পড়ুন — শুধু অক্ষর মনে রাখবেন না।

1. **ক** — DevSecOps মানে নিরাপত্তাকে শুরু থেকেই প্রতিটি ধাপে যুক্ত করা (shift-left); শেষে একবার অডিট করলে দেরি হয়ে যায়।
2. **খ** — ডিজাইন পর্যায়ে threat modeling করলে আর্কিটেকচার বদলানো সস্তা হয়; প্রোডাকশনে গিয়ে বদলানো ব্যয়বহুল।
3. **গ** — STRIDE-এ T হলো Tampering, অর্থাৎ ডেটা বা কোডের অননুমোদিত পরিবর্তন।
4. **ঘ** — পেমেন্ট API-তে প্রথমে প্রমাণীকরণ ও অনুমোদন বাইপাস এবং সংবেদনশীল ডেটা ফাঁসের হুমকি বিবেচনা করতে হয়, কারণ এর প্রভাব সবচেয়ে বেশি।
5. **গ** — OWASP Top 10:2021-এ A01 হলো Broken Access Control।
6. **ক** — সার্ভার-সাইডে প্রতিটি রিকোয়েস্টে deny-by-default অনুমোদন যাচাই করাই নির্ভরযোগ্য; ফ্রন্টএন্ডে লুকানো নিরাপত্তা নয়।
7. **ঘ** — Parameterized query / prepared statement ইনপুটকে ডেটা হিসেবে ধরে, কোড হিসেবে নয় — তাই SQL Injection আটকায়।
8. **খ** — সীমিত সময়ে শোষণযোগ্যতা ও প্রভাব অনুযায়ী অগ্রাধিকার ঠিক করতে হয়; প্রমাণীকরণ/অনুমোদন বাইপাস ও SQLi সাধারণত আগে আসে।
9. **খ** — আউটপুট এনকোডিং, CSP ও প্রসঙ্গ-সচেতন এস্কেপিং একসাথে XSS প্রতিরোধ করে; শুধু একটি শব্দ ব্লক করা যথেষ্ট নয়।
10. **গ** — SSRF-এ আক্রমণকারী সার্ভারকে দিয়ে অভ্যন্তরীণ সার্ভিস বা মেটাডেটা এন্ডপয়েন্টে রিকোয়েস্ট পাঠায়।
11. **ঘ** — সিক্রেট ফাঁস হলে প্রথমে revoke/rotate করতে হয়, তারপর ইতিহাস থেকে সরিয়ে প্রভাব যাচাই করতে হয়; শুধু .gitignore যোগ করলে পুরনো কমিটে কীটি থেকেই যায়।
12. **ক** — SAST কোড না চালিয়ে সোর্স কোডের ভেতরের দুর্বলতা পরীক্ষা করে।
13. **ঘ** — DAST চলমান অ্যাপকে বাইরে থেকে পরীক্ষা করে রানটাইম দুর্বলতা ও ভুল কনফিগারেশন ধরে।
14. **খ** — SCA থার্ড-পার্টি লাইব্রেরি ও ডিপেন্ডেন্সিতে জানা CVE ও লাইসেন্স ঝুঁকি খুঁজে বের করে।
15. **ক** — SAST বন্ধ না করে নিয়ম টিউন করে উচ্চ-আস্থার নিয়মে ব্লক করা ও বাকিগুলো warning রাখা এবং ডেভেলপারদের সাথে ত্রিয়াজ করাই সঠিক পথ।
16. **গ** — SBOM প্রতিটি উপাদান ও সংস্করণের তালিকা দেয়, তাই নতুন CVE এলে দ্রুত জানা যায় কোন সার্ভিস আক্রান্ত।
17. **গ** — SLSA সফটওয়্যার সাপ্লাই চেইনের অখণ্ডতা নিশ্চিত করে — বিল্ডের উৎস, প্রক্রিয়া ও আর্টিফ্যাক্ট যাচাইযোগ্য করে।
18. **ঘ** — Critical CVE পেলে বেস ইমেজ ও ডিপেন্ডেন্সি আপডেট করে আবার বিল্ড ও স্ক্যান করতে হয় এবং শোষণযোগ্যতা যাচাই করে অগ্রাধিকার ঠিক করতে হয়।
19. **ক** — Multi-stage build, minimal/distroless বেস ও অপ্রয়োজনীয় প্যাকেজ বাদ দিলে ইমেজ ছোট হয় ও আক্রমণ-ক্ষেত্র কমে।
20. **খ** — securityContext-এ runAsNonRoot ও readOnlyRootFilesystem দেওয়া এবং Pod Security Standards প্রয়োগ করলে Pod root হিসেবে চলতে পারে না।

21. **ঘ** — Least privilege মানে প্রত্যেককে শুধু তার কাজের জন্য প্রয়োজনীয় ন্যূনতম অনুমতি দেওয়া।
22. **ক** — Secret base64-এ এনকোড করে etcd-তে রাখা হয়, ডিফল্টে এনক্রিপ্টেড নয়, এবং env থেকে লগে ফাঁস হতে পারে — তাই external secrets manager ও encryption at rest দরকার।
23. **খ** — NetworkPolicy না থাকলে সব Pod একে অপরের সাথে অবাধে কথা বলতে পারে, ফলে lateral movement সহজ হয়।
24. **গ** — OPA/Kyverno ডেপ্লয়মেন্টের আগে নীতি (যেমন latest ট্যাগ নিষেধ, resource limit বাধ্যতামূলক) যাচাই করে নীতি-বিরোধী কাজ আটকে দেয়।
25. **ক** — CVSS ৯.৮ মানে Critical তীব্রতা — সহজে শোষণযোগ্য ও ব্যাপক প্রভাব।
26. **ঘ** — CVSS স্কোর বেশি হলেই প্রথমে ঠিক করতে হবে এমন নয়; শোষণযোগ্যতা, এক্সপোজার, ব্যবসায়িক প্রভাব ও compensating control মিলিয়ে ঝুঁকি-ভিত্তিক অগ্রাধিকার ঠিক করতে হয়।
27. **খ** — Vulnerability management-এর সঠিক ক্রম: আবিষ্কার → ত্রিয়াজ ও অগ্রাধিকার → প্রতিকার → যাচাই → পুনরাবৃত্তি।
28. **গ** — S3 বাকেট public হলে অবিলম্বে public অ্যাক্সেস বন্ধ করে লগ থেকে কে কী দেখেছে তা যাচাই করতে হয় এবং Block Public Access দিয়ে পুনরাবৃত্তি রোধ করতে হয়।
29. **খ** — CSPM ক্লাউড কনফিগারেশন ক্রমাগত স্ক্যান করে ভুল কনফিগারেশন, অতিরিক্ত অনুমতি ও কমপ্লায়েন্স বিচ্যুতি ধরিয়ে দেয়।
30. **ক** — IAM-এ নিরাপদ অভ্যাস: least privilege, MFA, স্বল্পমেয়াদি ক্রেডেনশিয়াল ও নিয়মিত অ্যাক্সেস রিভিউ।
31. **গ** — CI/CD-তে সিক্রেট মাস্কড ভেরিয়েবল বা secrets manager থেকে রানটাইমে আনতে হয়, লগে দেখানো যাবে না এবং স্বল্পমেয়াদি টোকেন ব্যবহার করতে হয়।
32. **ঘ** — সাপ্লাই চেইন আক্রমণ থেকে বাঁচতে পিন করা সংস্করণ, লকফাইল, SBOM, স্বাক্ষর যাচাই ও নির্ভরযোগ্য রেজিস্ট্রি একসাথে দরকার।
33. **ঘ** — PR-এর মাধ্যমে টোকেন চুরি করে রিপোজিটরি বা রিলিজে লেখার চেষ্টা হতে পারে, তাই GITHUB_TOKEN-এ ন্যূনতম অনুমতি দিতে হয়।
34. **গ** — Cosign দিয়ে স্বাক্ষর করলে ইমেজটি নির্ভরযোগ্য উৎস থেকে এসেছে ও পথে বদলায়নি — তা যাচাই করা যায়।
35. **ক** — Zero Trust-এর নীতি: কাউকে ডিফল্টে বিশ্বাস নয় — প্রতিটি অ্যাক্সেস যাচাই করো (never trust, always verify)।
36. **খ** — SRE সফটওয়্যার ইঞ্জিনিয়ারিং দিয়ে অপারেশন সমস্যার সমাধান করে নির্ভরযোগ্য সেবা দেয়।
37. **ক** — SLI হলো সেবার মানের পরিমাপযোগ্য সূচক, যেমন সফল রিকোয়েস্টের অনুপাত বা লেটেন্সি।
38. **খ** — SLO ৯৯.৯% মানে নির্ধারিত সময়ে ৯৯.৯% রিকোয়েস্ট লক্ষ্য মেনে (যেমন লেটেন্সি সীমার মধ্যে) সফল হতে হবে।
39. **ঘ** — Error budget হলো SLO-র অনুমোদিত ব্যর্থতার পরিমাণ (১০০% − SLO); বাজেট শেষ হলে ফিচার থামিয়ে নির্ভরযোগ্যতায় মনোযোগ দিতে হয়।
40. **গ** — Error budget শেষ হলে ফিচার ফ্রিজ ঘোষণা করে নির্ভরযোগ্যতা, টেকনিক্যাল ঋণ ও স্থিতিশীলতায় বিনিয়োগ করাই সঠিক সিদ্ধান্ত।
41. **খ** — Availability-র জন্য সফল রিকোয়েস্টের অনুপাতই সবচেয়ে উপযুক্ত SLI।
42. **ক** — SLO নির্ধারণে ব্যবহারকারীর প্রত্যাশা ও ব্যবসায়িক প্রভাবের সাথে সামঞ্জস্য জরুরি; খুব কঠিন SLO অপ্রয়োজনীয় খরচ বাড়ায়, খুব ঢিলা SLO ব্যবহারকারী হারায়।
43. **ঘ** — Golden Signals চারটি: Latency, Traffic, Errors, Saturation।
44. **গ** — RED মেথড রিকোয়েস্ট-ভিত্তিক সার্ভিসের স্বাস্থ্য বোঝায় — Rate, Errors, Duration।

45. **গ** — USE মেথড প্রতিটি রিসোর্সের Utilization, Saturation ও Errors বোঝায়।
46. **ঘ** — Alert fatigue কমাতে শুধু ব্যবহারকারীকে প্রভাবিত করে এমন, কার্যকর (actionable) অ্যালার্ট রাখতে হয়, সাথে থ্রেশহোল্ড ও runbook।
47. **খ** — যে অ্যালার্টে ব্যবহারকারী-প্রভাব নেই ও পদক্ষেপ স্পষ্ট নয়, সেটি টিউন বা সরিয়ে দেওয়া উচিত; non-actionable অ্যালার্ট আসল অ্যালার্ট মিস করায়।
48. **ক** — Prometheus counter-এর মান কেবল বাড়ে (বা রিসেটে শূন্য হয়), যেমন মোট রিকোয়েস্ট সংখ্যা।
49. **ঘ** — Histogram বিতরণ দেখায়, তাই p50/p95/p99 বের করা যায় এবং গড়ে লুকিয়ে থাকা tail latency ধরা যায়।
50. **ক** — OpenTelemetry ভেন্ডর-নিরপেক্ষভাবে trace, metric ও log সংগ্রহ ও পাঠানোর মান (standard) দেয়।
51. **খ** — Span হলো একটি রিকোয়েস্টের ভেতরে একটি একক কাজের সময়কালসহ রেকর্ড; একাধিক span মিলে একটি trace তৈরি হয়।
52. **গ** — লগ, মেট্রিক ও ট্রেস একসাথে থাকলে কখন (metric), কোথায় (trace) ও কেন (log) — তিনটি প্রশ্নের উত্তর মেলে।
53. **ক** — ইনসিডেন্টের প্রথম ৫ মিনিটে প্রভাব নির্ণয়, যোগাযোগ শুরু ও ব্যবহারকারীর ক্ষতি কমানো (mitigate) সবচেয়ে গুরুত্বপূর্ণ; রুট কজ পরে।
54. **গ** — Blameless postmortem-এ দোষ না খুঁজে সিস্টেমের দুর্বলতা ও প্রক্রিয়াগত কারণ খোঁজা হয়, তাতে সত্য তথ্য বের হয় ও পুনরাবৃত্তি রোধ হয়।
55. **ঘ** — SEV-1 মানে গুরুতর সেবা ব্যাহত — বেশিরভাগ ব্যবহারকারী আক্রান্ত বা ডেটা ক্ষতির ঝুঁকি, অবিলম্বে সাড়া প্রয়োজন।
56. **খ** — MTTR কমাতে রানবুক, অনুশীলিত DR/রোলব্যাক পরিকল্পনা ও স্পষ্ট ভূমিকা (Incident Commander, comms) সবচেয়ে কার্যকর।
57. **খ** — 5 Whys-এ বারবার "কেন" জিজ্ঞাসা করে লক্ষণের পেছনের গভীর কারণে পৌঁছানো হয়।
58. **ঘ** — ডেপ্লয়ের পরপরই error rate বাড়লে প্রথমে দ্রুত রোলব্যাক করে সেবা স্থিতিশীল করতে হয়, তারপর শান্তভাবে কারণ বিশ্লেষণ করতে হয়।
59. **গ** — Canary deployment-এ অল্প সংখ্যক ব্যবহারকারীর কাছে নতুন সংস্করণ পরীক্ষা করে ঝুঁকি সীমিত রাখা যায় ও সমস্যা হলে সহজে ফেরা যায়।
60. **ক** — Blue-Green deployment-এ দুটি পরিবেশ (Blue = বর্তমান, Green = নতুন) রেখে ট্রাফিক সুইচ করা হয়; রোলব্যাক মানে ট্রাফিক ফিরিয়ে দেওয়া।

উত্তরমালা • MCQ ৬১–১২০ + মডেল উত্তর ও মার্কিং নোট

উত্তর নির্দেশিকা — MCQ ৬১–১২০ ও মডেল উত্তর

১. MCQ ৬১–১২০-এর উত্তরমালা

1. **গ** — Load test প্রত্যাশিত লোডে আচরণ যাচাই করে; stress test সীমার বাইরে চাপ দিয়ে ভাঙন-বিন্দু ও পুনরুদ্ধার যাচাই করে।
2. **ক** — ক্যাপাসিটি প্ল্যানিং-এ ঐতিহাসিক ট্রাফিক, বৃদ্ধির হার, পিক/সিজনালিটি ও ব্যবসায়িক পরিকল্পনা লাগে; শুধু বর্তমান CPU দেখলে হবে না।
3. **খ** — HPA CPU/মেমোরি ব্যবহার বা কাস্টম মেট্রিকের ভিত্তিতে Pod-এর সংখ্যা বাড়ায়-কমায়।
4. **ঘ** — ডেটাবেস CPU ১০০% হলে Read replica/ক্যাশ দিয়ে রিড লোড কমাতে হয়, ধীর কোয়েরি খুঁজতে হয় ও rate limiting/load shedding দিয়ে ডেটাবেসকে রক্ষা করতে হয়।
5. **ঘ** — RTO হলো বিপর্যয়ের পর সেবা পুনরুদ্ধারে সর্বোচ্চ গ্রহণযোগ্য সময়।
6. **খ** — RPO হলো সর্বোচ্চ গ্রহণযোগ্য ডেটা ক্ষতি — যেমন ১৫ মিনিটের ডেটা হারানো যাবে।
7. **গ** — RTO ১ ঘণ্টা ও RPO ১৫ মিনিটের জন্য নিয়মিত স্বয়ংক্রিয় ব্যাকআপ, পয়েন্ট-ইন-টাইম রিকভারি ও নিয়মিত restore পরীক্ষা দরকার।
8. **ক** — অপরীক্ষিত ব্যাকআপ প্রায়শই কাজ করে না — করাপ্ট, অসম্পূর্ণ বা ভুল কনফিগারেশনের কারণে; restore পরীক্ষাই প্রমাণ করে ব্যাকআপ সত্যিই কাজ করে।
9. **ক** — Chaos engineering-এর উদ্দেশ্য ইচ্ছাকৃতভাবে ছোট, নিয়ন্ত্রিত ব্যর্থতা ঘটিয়ে সিস্টেমের দুর্বলতা ও পুনরুদ্ধার ক্ষমতা আগেভাগে জানা।
10. **গ** — Chaos experiment-এর আগে হাইপোথিসিস, blast radius সীমিত করা, abort শর্ত, পর্যবেক্ষণ ও রোলব্যাক পরিকল্পনা ঠিক করতে হয়।
11. **ঘ** — FinOps-এর লক্ষ্য ক্লাউড খরচে স্বচ্ছতা, দায়বদ্ধতা ও মূল্য — প্রয়োজনীয় খরচে সেরা ফল পাওয়া, শুধু খরচ কমানো নয়।
12. **খ** — বিল বাড়লে প্রথমে খরচের ভাঙন দেখে কোন সার্ভিস/রিসোর্স/ট্যাগে বৃদ্ধি, idle রিসোর্স ও অপ্রত্যাশিত ডেটা ট্রান্সফার খুঁজতে হয়।
13. **গ** — Idle রিসোর্স কমাতে ব্যবহারের মেট্রিক দেখে অব্যবহৃত/কম-ব্যবহৃত রিসোর্স right-size করা বা বন্ধ করা ও অটো-শাটডাউন নীতি করতে হয়।
14. **ক** — Policy as Code-তে নীতি কোড হিসেবে সংস্করণ-নিয়ন্ত্রিত, পরীক্ষাযোগ্য ও স্বয়ংক্রিয়ভাবে প্রয়োগযোগ্য হয়।
15. **ঘ** — SOC 2/ISO 27001-এ সবচেয়ে গুরুত্বপূর্ণ প্রমাণ হলো নিয়ন্ত্রণ আসলেই চলছে তার প্রমাণ — লগ, টিকিট, রিভিউ রেকর্ড ও ধারাবাহিক প্রয়োগ।
16. **খ** — Vault কেন্দ্রীয়ভাবে সিক্রেট সংরক্ষণ করে, স্বল্পমেয়াদি (dynamic) সিক্রেট দেয়, অডিট লগ ও অ্যাক্সেস নিয়ন্ত্রণ দেয়।
17. **খ** — TLS সার্টিফিকেট মেয়াদোত্তীর্ণ হলে ব্রাউজার সতর্কবার্তা দেয় ও সংযোগ অবিশ্বস্ত দেখায়; স্বয়ংক্রিয় নবায়ন ও মেয়াদ-অ্যালার্ট জরুরি।
18. **গ** — WAF HTTP স্তরে সাধারণ আক্রমণ (SQLi, XSS প্যাটার্ন) ফিল্টার করে — তবে এটি নিরাপদ কোডের বিকল্প নয়।
19. **ক** — Rate limiting অপব্যবহার, ব্রুট-ফোর্স ও হঠাৎ লোড থেকে সেবাকে রক্ষা করে।

20. **ঘ** — ব্রুট-ফোর্সে rate limiting, অ্যাকাউন্ট লকআউট/ক্যাপচা, MFA উৎসাহিত করা ও সন্দেহজনক IP পর্যবেক্ষণ করা একসাথে করতে হয়।
21. **ঘ** — Log retention-এ আইনি/কমপ্লায়েন্স প্রয়োজন, তদন্তের সময়কাল, স্টোরেজ খরচ ও সংবেদনশীল ডেটা মাস্কিং বিবেচনা করতে হয়।
22. **গ** — Observability-র তিনটি স্তম্ভ: Logs, Metrics, Traces।
23. **খ** — Burn-rate অ্যালার্ট error budget কত দ্রুত খরচ হচ্ছে তা দেখায় ও ব্যবহারকারীর অভিজ্ঞতার সাথে সরাসরি যুক্ত, তাই অপ্রয়োজনীয় অ্যালার্ট কমে।
24. **ক** — ৯৯.৯৯% SLO-র ৩০ দিনের error budget ৪ মিনিট ১৯ সেকেন্ড (৩০ দিন × ০.০১% = ৪.৩২ মিনিট)।
25. **ক** — ৯৯.৯% SLO-র ৩০ দিনের error budget ৪৩ মিনিট ১২ সেকেন্ড (৩০ দিন × ০.১% = ৪৩.২ মিনিট)।
26. **ঘ** — p95 বেড়ে গেলেও গড় একই থাকার মানে অল্প কিছু রিকোয়েস্ট (tail) অনেক ধীর হচ্ছে; গড় তা লুকিয়ে রাখে, tail latency তদন্ত দরকার।
27. **গ** — Connection pool শেষ হলে রিকোয়েস্ট আটকে যায় ও timeout বাড়ে, যদিও অ্যাপ সার্ভার সুস্থ থাকে।
28. **খ** — Circuit breaker ব্যর্থ ডিপেন্ডেন্সিতে বারবার রিকোয়েস্ট পাঠিয়ে তাকে আরও চাপ দেওয়া থেকে বিরত রাখে ও দ্রুত fail-fast করে।
29. **খ** — Retry-তে exponential backoff + jitter একসাথে অনেক retry (thundering herd) এড়িয়ে ব্যর্থ সার্ভিসকে শ্বাস নেওয়ার সুযোগ দেয়।
30. **ক** — Feature flag দিয়ে কোড ডেপ্লয় না করে ঝুঁকিপূর্ণ ফিচার দ্রুত বন্ধ (kill switch) করা যায়।
31. **গ** — Immutable infrastructure মানে সার্ভারে প্যাচ দিয়ে বদল না করে নতুন ইমেজ/সার্ভার দিয়ে প্রতিস্থাপন করা।
32. **ঘ** — IaC স্ক্যান করলে Terraform/Helm ফাইলে ভুল কনফিগারেশন (যেমন public বাকেট, খোলা পোর্ট) ডেপ্লয়ের আগেই ধরা যায়।
33. **গ** — GitOps-এ গিটকে সত্যের একমাত্র উৎস ধরে স্বয়ংক্রিয়ভাবে ক্লাস্টারের অবস্থা গিটের সাথে মেলানো হয়।
34. **ঘ** — হটফিক্সে ঝুঁকি নথিভুক্ত করে অনুমোদিত জরুরি পথে ন্যূনতম পরীক্ষাসহ ডেপ্লয় করতে হয় এবং পরে পূর্ণ পাইপলাইন দিয়ে যাচাই ও নথিভুক্ত করতে হয়।
35. **ক** — Data breach সন্দেহ হলে প্রথমে প্রভাব সীমিত (contain) করতে হয়, প্রমাণ সংরক্ষণ করতে হয়, ইনসিডেন্ট প্রক্রিয়া শুরু করতে হয় ও আইনি/কমপ্লায়েন্স দলকে জানাতে হয়।
36. **খ** — সব ডেভেলপারকে প্রোডাকশন ডেটাবেসে admin অ্যাক্সেস দেওয়া least privilege লঙ্ঘনের উদাহরণ।
37. **ক** — Container escape মানে কনটেইনার থেকে বের হয়ে হোস্ট সিস্টেমে অ্যাক্সেস পাওয়া — যেমন privileged কনটেইনার বা ভুল কনফিগারেশনের মাধ্যমে।
38. **খ** — Admission controller Kubernetes-এ রিসোর্স তৈরি/পরিবর্তনের অনুরোধ নীতি অনুযায়ী যাচাই করে অনুমোদন বা প্রত্যাখ্যান করে।
39. **ঘ** — CVE-তে ব্যবহৃত ফাংশন ব্যবহার না করলেও reachability যাচাই করে ঝুঁকি নথিভুক্ত করতে হয় ও আপডেটের পরিকল্পনা করতে হয়, কারণ ভবিষ্যতে ব্যবহার হতে পারে।
40. **গ** — Toil কমালে ইঞ্জিনিয়াররা উদ্ভাবন ও নির্ভরযোগ্যতার কাজে সময় পান; toil বাড়লে বার্নআউট ও ভুল বাড়ে।
41. **ঘ** — RTO/RPO কিছুটা ঢিলা হলে ও খরচ নিয়ন্ত্রণে রাখতে চাইলে active-passive যথেষ্ট — passive অঞ্চল প্রস্তুত রেখে প্রয়োজনে ফেইলওভার করা যায়।
42. **গ** — Health check ভুল থাকলে অসুস্থ ইনস্ট্যান্সে ট্রাফিক যেতে থাকে বা সুস্থ ইনস্ট্যান্স সরিয়ে নেওয়া হয় — দুটোই বিপজ্জনক।

43. **ক** — Defence in depth-এর উদাহরণ: WAF + নিরাপদ কোড + least-privilege IAM + মনিটরিং — একাধিক স্তরে প্রতিরক্ষা, যাতে একটি স্তর ভাঙলেও অন্যটি রক্ষা করে।
44. **খ** — Security champion প্রোগ্রামে প্রতিটি ডেভ টিমে একজন নিরাপত্তা-সচেতন সদস্য রেখে নিরাপত্তা চর্চা দলে ছড়িয়ে দেওয়া হয়।
45. **খ** — DORA-তে পরিবর্তনের গতি মাপে deployment frequency — কত ঘন ঘন সফলভাবে ডেপ্লয় হয়।
46. **ঘ** — Change failure rate বেশি মানে অনেক পরিবর্তনই ইনসিডেন্ট বা রোলব্যাক ঘটাচ্ছে — পরীক্ষা, রিভিউ বা ক্যানারি প্রক্রিয়ায় দুর্বলতা আছে।
47. **ক** — ভালো SLO ব্যবহারকারীর অভিজ্ঞতার সাথে যুক্ত, পরিমাপযোগ্য, অর্জনযোগ্য ও নির্দিষ্ট সময়সীমাসহ হয়।
48. **গ** — ইনসিডেন্টে যোগাযোগে নিয়মিত, সৎ ও সংক্ষিপ্ত আপডেট দিতে হয় — প্রভাব, কী করা হচ্ছে ও পরের আপডেট কখন — একটি নির্দিষ্ট চ্যানেলে।
49. **গ** — Post-incident review-এর সবচেয়ে মূল্যবান আউটপুট হলো সুনির্দিষ্ট, দায়িত্বপ্রাপ্ত ও সময়সীমাসহ action items — যা সিস্টেম ও প্রক্রিয়া বদলায়।
50. **ক** — সঠিক backup কৌশল হলো 3-2-1 নিয়ম — তিন কপি, দুই ধরনের মাধ্যমে, একটি অফসাইট/অপরিবর্তনীয় কপি — ও নিয়মিত restore পরীক্ষা।
51. **ঘ** — Liveness বলে Pod বেঁচে আছে কি না (ব্যর্থ হলে রিস্টার্ট); readiness বলে Pod ট্রাফিক নিতে প্রস্তুত কি না (না হলে ট্রাফিক সরানো হয়, রিস্টার্ট নয়)।
52. **খ** — Error budget শেষ হলে তথ্য (burn rate, ব্যবহারকারী-প্রভাব, ঝুঁকি) দিয়ে ব্যবসার সাথে আলোচনা করে ফ্রিজ বা সীমিত রিলিজের সিদ্ধান্ত নিতে হয়।
53. **ঘ** — Supply-chain আক্রমণ মানে বিশ্বস্ত প্যাকেজ বা বিল্ড টুলে ক্ষতিকর কোড ঢুকিয়ে অনেক ভুক্তভোগীর কাছে পৌঁছে দেওয়া।
54. **ক** — নিরাপদ API ডিজাইনে সব এন্ডপয়েন্টে প্রমাণীকরণ ও অনুমোদন, ইনপুট যাচাই, rate limiting ও সংবেদনশীল ডেটা ফাঁস না করা দরকার।
55. **খ** — Monitoring জানা সমস্যার জন্য আগে থেকে নির্ধারিত মেট্রিক দেখে; observability অজানা সমস্যার কারণ খুঁজতে সিস্টেমের ভেতরের অবস্থা জিজ্ঞাসা করার ক্ষমতা দেয়।
56. **গ** — Chaos engineering-এর উপযুক্ত প্রথম পরীক্ষা: স্টেজিংয়ে একটি Pod বন্ধ করে সার্ভিস নিজে থেকে পুনরুদ্ধার হয় কি না দেখা — ছোট blast radius দিয়ে শুরু করা।
57. **ক** — FinOps-এ tagging গুরুত্বপূর্ণ কারণ কোন টিম/প্রজেক্ট/পরিবেশের জন্য কত খরচ হচ্ছে তা জানা যায় — দায়বদ্ধতা ও অপ্টিমাইজেশনের ভিত্তি।
58. **গ** — Shift-left security-র উদাহরণ: IDE-তে লিন্টার/সিক্রেট স্ক্যান, PR-এ SAST/SCA ও IaC স্ক্যান — ডেভেলপমেন্টের শুরুতেই নিরাপত্তা ধরা।
59. **খ** — ডেটা ফাঁসের আশঙ্কায় প্রথম অগ্রাধিকার: প্রমাণ সংরক্ষণ ও প্রভাব সীমিত করে কারা, কী ডেটা ও কতটা — তা নির্ণয় করা এবং আইনি/নিরাপত্তা দলকে জানানো।
60. **ঘ** — ভালো DevSecOps/SRE ইঞ্জিনিয়ারের সবচেয়ে গুরুত্বপূর্ণ গুণ: ঝুঁকি, ট্রেড-অফ ও ব্যবহারকারী-প্রভাব বুঝে নিরাপত্তা, গতি ও নির্ভরযোগ্যতার ভারসাম্য রেখে সিদ্ধান্ত নেওয়া ও তা স্পষ্টভাবে যোগাযোগ করা।

২. সংক্ষিপ্ত প্রশ্নের মডেল উত্তর (সংক্ষেপে)

- **১–২:** Shift-left মানে নিরাপত্তা শুরুতে আনা; threat modeling-এর চার প্রশ্ন: আমরা কী তৈরি করছি? কী ভুল হতে পারে? কী করব? কাজটি হয়েছে কি না যাচাই করেছি?

- **৩–৫:** Broken Access Control = অনুমতি ছাড়া অন্যের ডেটা দেখা; Cryptographic Failures = দুর্বল/অনুপস্থিত এনক্রিপশন। SAST কোডে, DAST চলমান অ্যাপে, SCA ডিপেন্ডেন্সিতে। SBOM থাকলে Log4j-এর মতো CVE-তে আক্রান্ত সার্ভিস মিনিটে চেনা যায়।
- **৬–৯:** SLSA L3-তে স্বাক্ষরিত provenance, বিচ্ছিন্ন (isolated) বিল্ড ও যাচাইযোগ্য উৎস দরকার। Distroless ইমেজ ছোট ও নিরাপদ, তবে ডিবাগ কঠিন। Pod Security: Privileged (অনিয়ন্ত্রিত), Baseline (ন্যূনতম সীমা), Restricted (কঠোরতম)। OPA-তে নীতি কোড হিসেবে লেখা যায়, যেমন "latest ট্যাগ নিষেধ"।
- **১০–১৪:** CVSS Base = দুর্বলতার নিজস্ব বৈশিষ্ট্য, Temporal = সময়ের সাথে বদল (exploit আছে কি না), Environmental = আপনার পরিবেশের প্রেক্ষাপট। ঝুঁকি-ভিত্তিক অগ্রাধিকারে স্কোর + শোষণযোগ্যতা + এক্সপোজার দেখতে হয়। ক্লাউডে public বাকেট, খোলা পোর্ট, অতিরিক্ত IAM অনুমতি — প্রতিকারে least privilege ও CSPM। Secrets rotation স্বয়ংক্রিয় হলে ফাঁসের জানালা (window) ছোট হয়। Secure CI/CD-তে SAST/SCA, স্বাক্ষর, সিক্রেট মাস্কিং, অনুমোদন ও অডিট লগ থাকা চাই।
- **১৫–১৯:** SLI = পরিমাপ, SLO = লক্ষ্য, SLA = চুক্তি। ৯৯.৯% SLO-র ৩০ দিনের বাজেট ৪৩ মিনিট ১২ সেকেন্ড। Golden Signals = latency, traffic, errors, saturation; RED = rate, errors, duration। Actionable alert-এ স্পষ্ট প্রভাব, স্পষ্ট পদক্ষেপ ও runbook থাকে। Trace ID প্রতিটি span-এ একই থাকে বলে পুরো রিকোয়েস্টের পথ অনুসরণ করা যায়।
- **২০–২৪:** Blameless postmortem দোষ না খুঁজে সিস্টেমের কারণ খোঁজে। Incident Commander সমন্বয়, সিদ্ধান্ত ও যোগাযোগের দায়িত্বে থাকেন। 5 Whys-এ বারবার "কেন" জিজ্ঞাসা করে গভীর কারণে পৌঁছানো হয়। Canary = ঝুঁকি কমাতে, Blue-Green = দ্রুত রোলব্যাকে, Rolling = ধীরে ধীরে বদলে। Load/stress test থেকে ভাঙন-বিন্দু জেনে headroom রেখে ক্যাপাসিটি ঠিক করতে হয়।
- **২৫–৩০:** RTO = কতক্ষণে ফিরবে, RPO = কত ডেটা হারানো যাবে। Chaos পরীক্ষায় hypothesis, ছোট blast radius ও abort শর্ত থাকা চাই। Tagging ছাড়া খরচ কার তা জানা যায় না। Toil = পুনরাবৃত্তিমূলক ম্যানুয়াল কাজ; অটোমেশন (যেমন স্বয়ংক্রিয় সার্টিফিকেট নবায়ন) তা কমায়। Defence in depth = একাধিক স্তরে প্রতিরক্ষা (WAF, নিরাপদ কোড, IAM, মনিটরিং)।

৩. পরিস্থিতি ও কেস স্টাডি মূল্যায়ন নির্দেশিকা

- **ভালো উত্তরে থাকবে:** সম্ভাব্য কারণের তালিকা (অনুমান নয়, প্রমাণ দিয়ে যাচাই), যাচাইয়ের ক্রম (সবচেয়ে সস্তা ও নিরাপদ পরীক্ষা আগে), তাৎক্ষণিক প্রশমন (ব্যবহারকারীর ক্ষতি কমানো) ও দীর্ঘমেয়াদি প্রতিকার (প্রক্রিয়া/অটোমেশন)।
- **দুর্বল উত্তর:** শুধু একটি কারণ ধরে নেওয়া, "সার্ভার রিস্টার্ট করুন" বলে শেষ করা, বা ট্রেড-অফ না লেখা।
- **কেস স্টাডিতে খুঁজুন:** কোন নিয়ন্ত্রণ ব্যর্থ হয়েছে, কোনটি থাকলে ধরা যেত, এবং action item-টি সুনির্দিষ্ট, দায়িত্বপ্রাপ্ত ও সময়সীমাসহ কি না।

৪. ফাইনাল পরীক্ষার মার্কিং নোট

- অংশ ক (MCQ): প্রতিটি সঠিক উত্তরে ১ নম্বর।
- অংশ খ (সংক্ষিপ্ত): সংজ্ঞা ১, উদাহরণ ১, যুক্তি/প্রয়োগ ২ — মোট ৪।
- অংশ গ (পরিস্থিতি): কারণ ৩, যাচাই ২, প্রশমন ২, দীর্ঘমেয়াদি প্রতিকার ৩ — মোট ১০।
- অংশ ঘ (ল্যাব): কাজের সম্পূর্ণতা, আউটপুটের সঠিকতা ও প্রতিবেদনের স্পষ্টতা — টেবিলের নম্বর অনুযায়ী। আংশিক কাজে আংশিক নম্বর দিন।

৫. ক্যাপস্টোন রুব্রিক

মাত্রা	দুর্বল (১)	ভালো (৩)	চমৎকার (৫)
নিরাপত্তা নকশা	শুধু একটি নিয়ন্ত্রণ	একাধিক স্তরে নিয়ন্ত্রণ, কিছু যুক্তি	Threat model + defence in depth + যাচাইযোগ্য নিয়ন্ত্রণ
নির্ভরযোগ্যতা	কোনো SLO/RTO নেই	SLO ও রোলব্যাক পরিকল্পনা আছে	SLO, error budget, DR পরীক্ষাসহ সম্পূর্ণ পরিকল্পনা
পর্যবেক্ষণযোগ্যতা	শুধু লগ	মেট্রিক + অ্যালার্ট	লগ/মেট্রিক/ট্রেস + burn-rate অ্যালার্ট + runbook
প্রয়োগ ও যুক্তি	সিদ্ধান্তের কারণ নেই	কিছু ট্রেড-অফ লেখা	ঝুঁকি, খরচ ও ব্যবহারকারী-প্রভাবসহ স্পষ্ট ট্রেড-অফ
উপস্থাপনা	অগোছালো	পরিষ্কার, কিছু প্রমাণ	পরিষ্কার, প্রমাণসহ, পুনরুৎপাদনযোগ্য

মোট ২৫-এর মধ্যে ১৫ পেলে পাস, ২০+ পেলে distinction।